Implementing a CI/CD Pipeline for GlobalMart E-Commerce Platform

Overview of Project

Scenario:

GlobalMart, a leading e-commerce retailer with 300+ employees across 3 regions, has implemented a CI/CD pipeline for their product catalog service, but is experiencing deployment failures and site outages. With an estimated $25,000 in lost revenue per hour of downtime, they urgently need help troubleshooting their pipeline issues. As their Cloud Support Engineer, you'll identify and resolve the misconfiguration problems causing these failures.

Solution:

A fully automated CI/CD pipeline using AWS CodePipeline and CodeDeploy with comprehensive monitoring capabilities. This project demonstrates how AWS DevOps services work together and teaches essential troubleshooting skills for common CI/CD misconfigurations.

About Project:

I recreated GlobalMart's CI/CD pipeline with the same issues their team faced, then troubleshot using CloudWatch, CodePipeline logs, and the AWS Console. I gained hands-on experience diagnosing and fixing real-world deployment issues — just as a cloud support engineer would in a production environment.

Steps Performed:

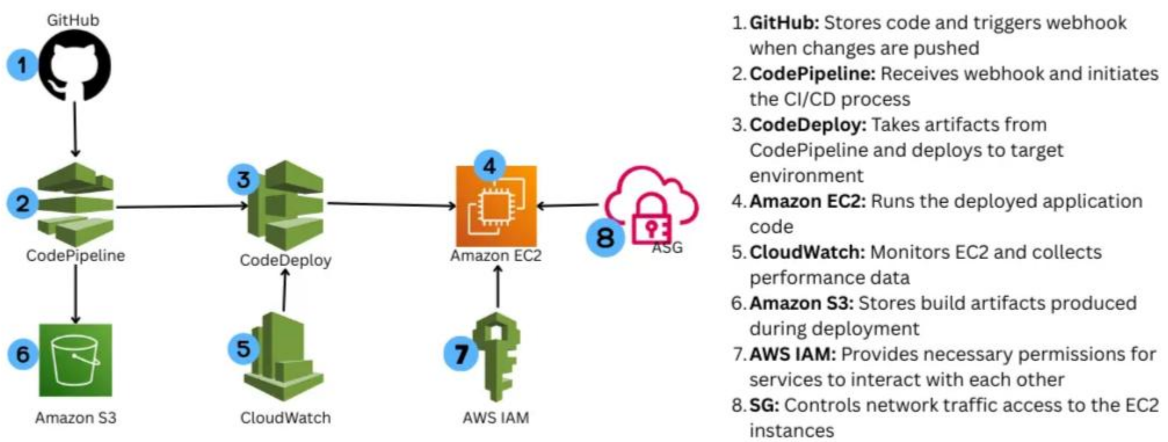
1. Setting Up Environment: Prepared EC2 and GitHub repository
2. Configuring AWS Services: Setting up IAM roles and deployment tools

3. Building CI/CD Pipeline: Configuring CodeDeploy and CodePipeline
4. Troubleshooting Pipeline Issues: Fixed common deployment problems

Services Used:

- AWS CodePipeline: Fully managed continuous delivery service
- AWS CodeDeploy: Deployment service for EC2 instances
- Amazon EC2: Compute instances for hosting the e-commerce application
- AWS IAM: Identity and access management for AWS resources
- GitHub: Source code repository with webhook integration
- Amazon S3: Storage for deployment artifacts
- AWS Security Groups: Virtual firewalls controlling network traffic

This is the architectural diagram for the project:



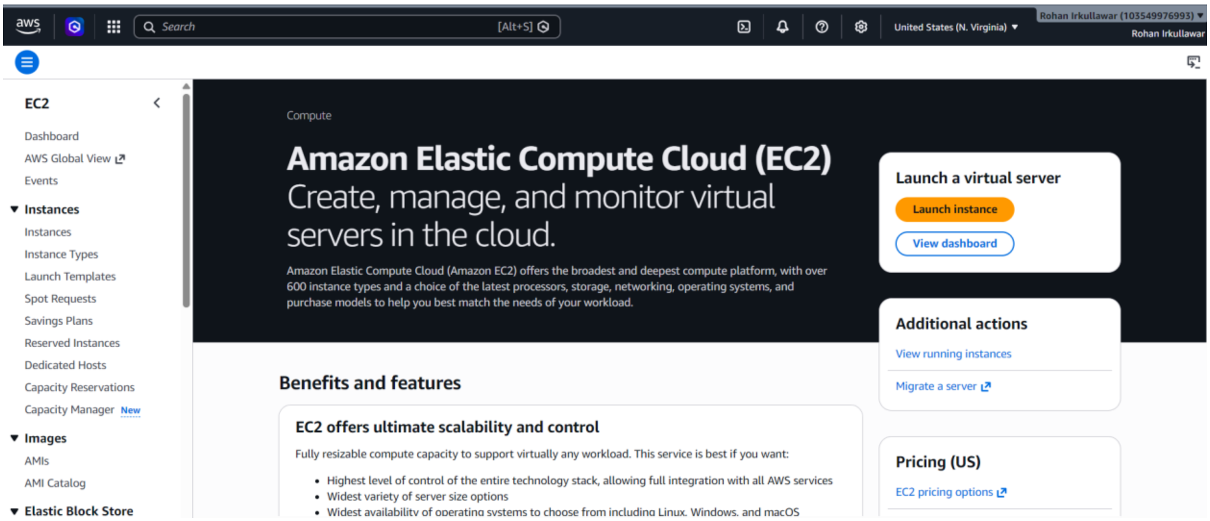
Infrastructure Setup: EC2 Configuration

Introduction:

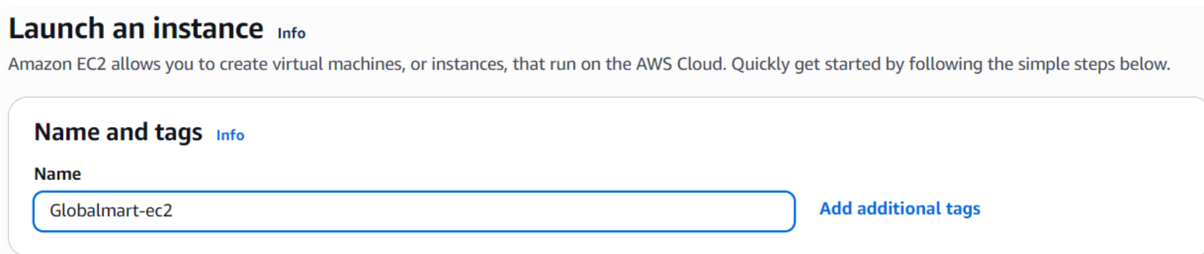
In this hands-on project, I built, intentionally broke, and fixed a CI/CD pipeline for GlobalMart's e-commerce catalog. This first section walks through how I created a fully functional pipeline that deploys a React application to an EC2 instance.

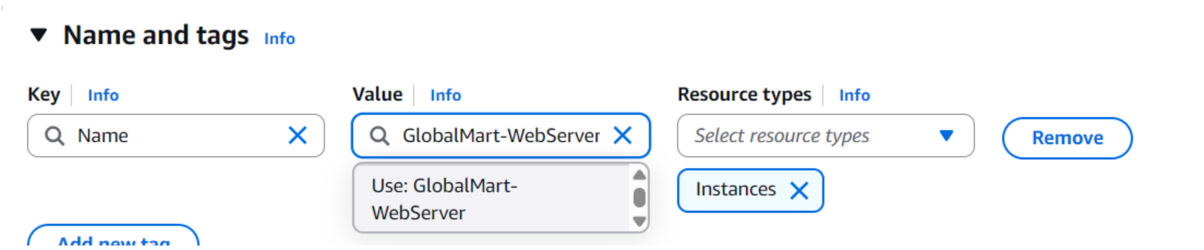
Implementation Steps

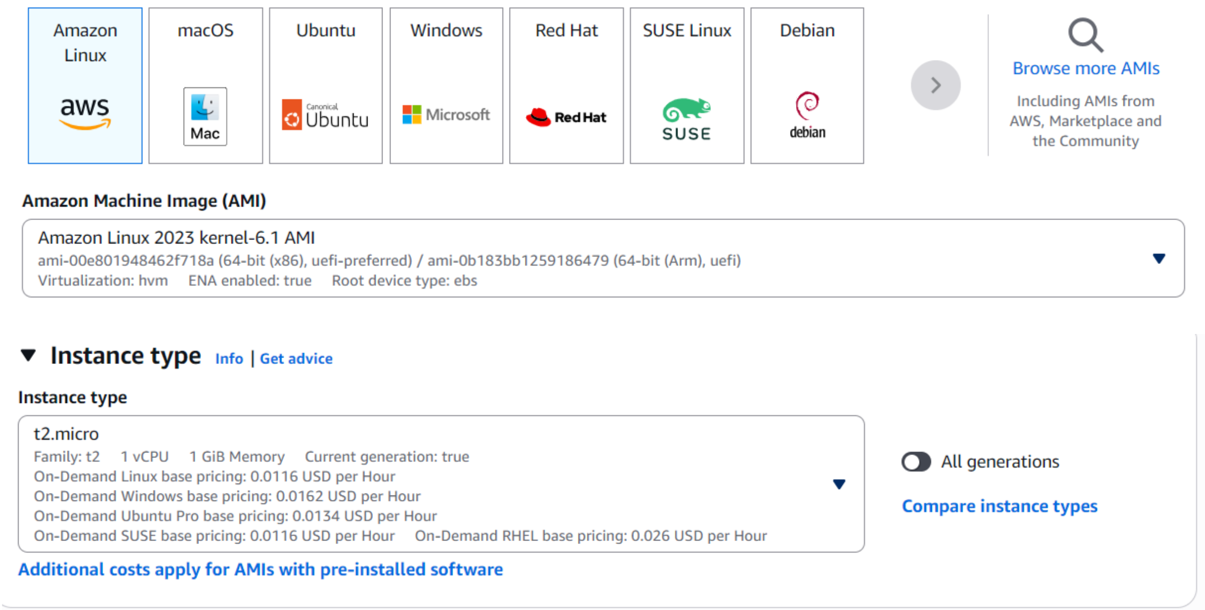
Launched Instance:

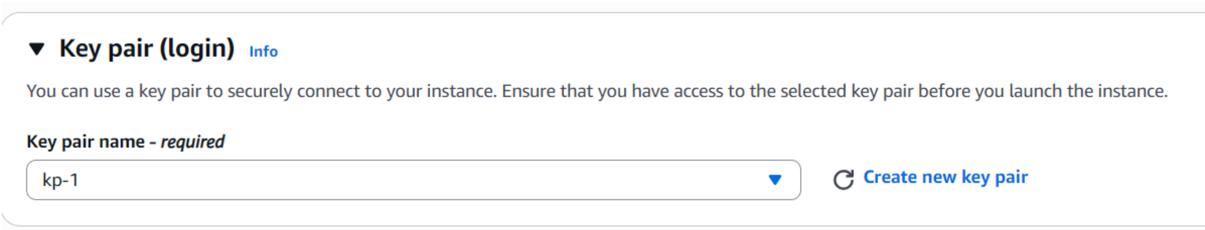


Configured instance:



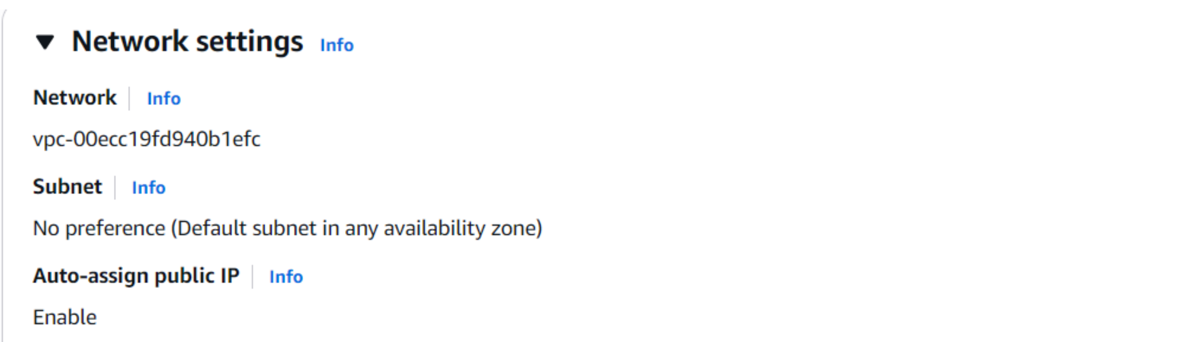




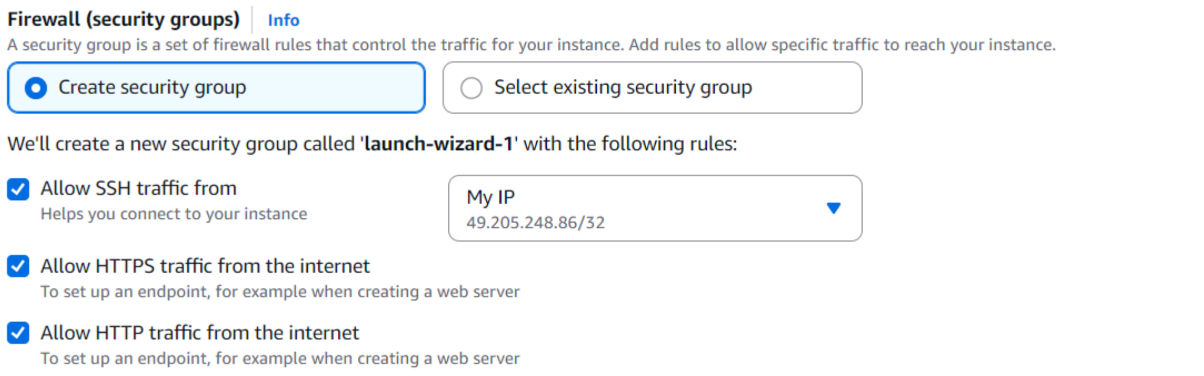


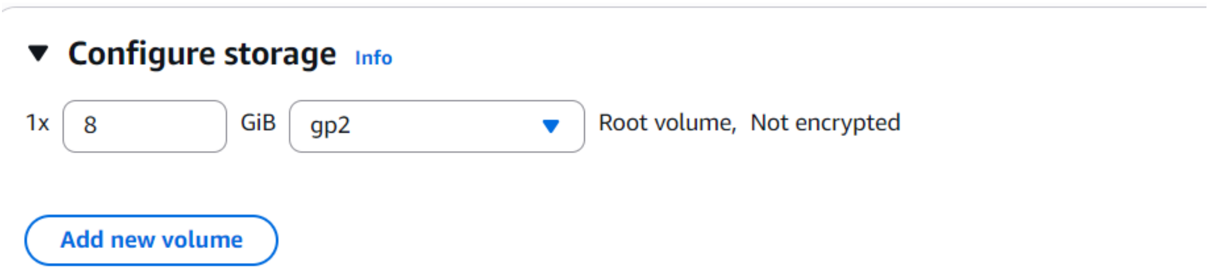
Configured Network Settings:

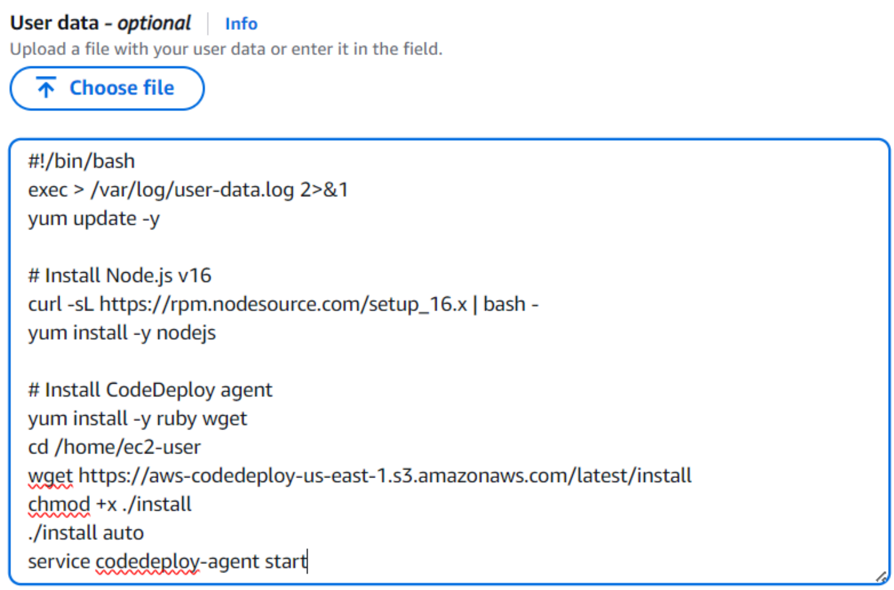
- Used the default VPC and subnet
- Ensured "Auto-assign public IP" is set to "Enable"



Setting up Security Group:

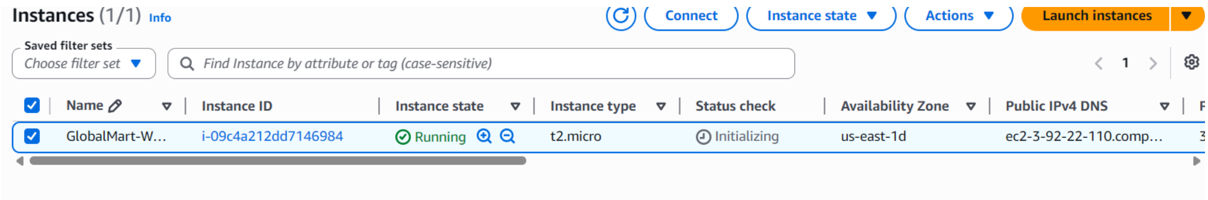




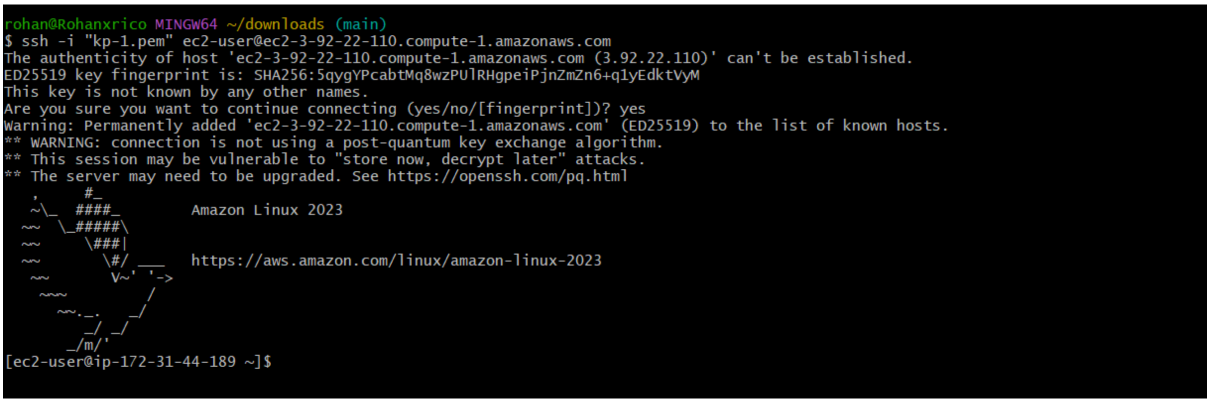


This bootstrap script automatically runs when instance launches, installing required software without manual intervention. This is a key concept in Infrastructure as Code (IaC) and enables consistent, repeatable deployments. **The script installs:** Node.js: The runtime environment needed to run our JS-based application

CodeDeploy agent: Required for AWS CodeDeploy to deploy applications to this instance

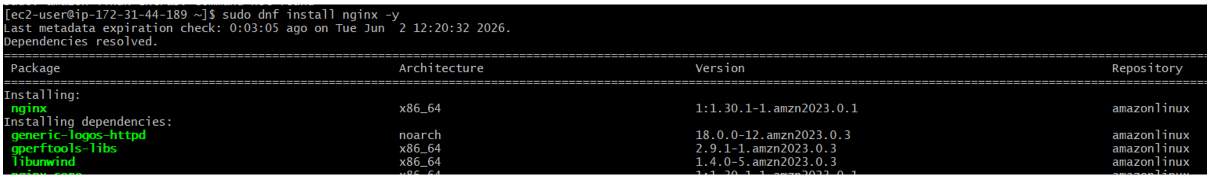


Connected via SSH:

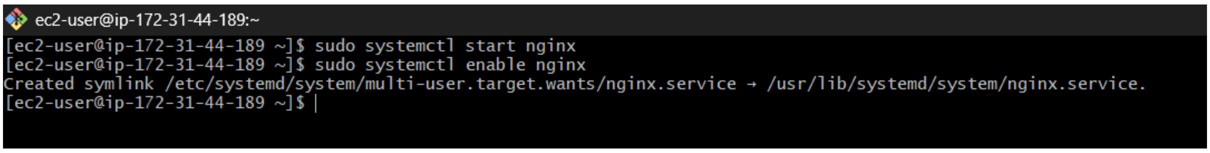


```
rohan@Rohanxrico MINGW64 ~/downloads (main)
$ ssh -i "kp-1.pem" ec2-user@ec2-3-92-22-110.compute-1.amazonaws.com
The authenticity of host 'ec2-3-92-22-110.compute-1.amazonaws.com (3.92.22.110)' can't be established.
ED25519 key fingerprint is: SHA256:5qygYPcabtMq8wzPUlRHgpeiPjnZmZn6+q1yEdktVyM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-3-92-22-110.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-44-189 ~]$
```

Installed Nginx:

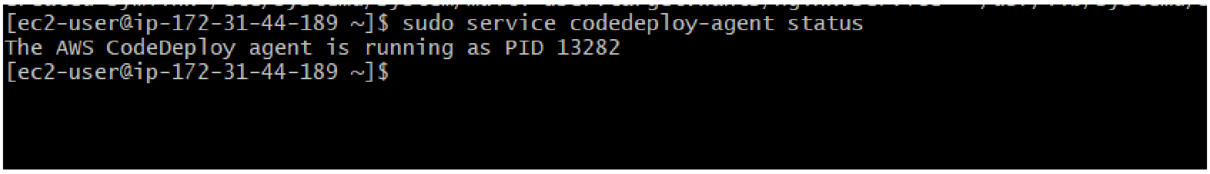


```
[ec2-user@ip-172-31-44-189 ~]$ sudo dnf install nginx -y
Last metadata expiration check: 0:03:05 ago on Tue Jun  2 12:20:32 2026.
Dependencies resolved.
================================================================================
 Package                Architecture        Version                     Repository
================================================================================
Installing:
 nginx                  x86_64              1:1.30.1-1.amzn2023.0.1     amazonlinux
Installing dependencies:
 generic-logos-httpd    noarch              18.0.0-12.amzn2023.0.3      amazonlinux
 gperftools-libs        x86_64              2.9.1-1.amzn2023.0.3        amazonlinux
 libunwind              x86_64              1.4.0-5.amzn2023.0.3        amazonlinux
```

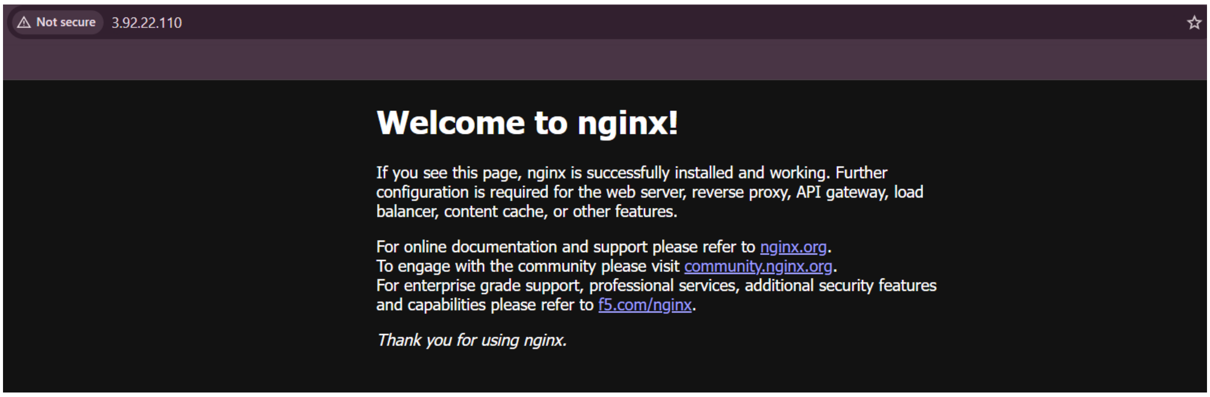


```
ec2-user@ip-172-31-44-189:~
[ec2-user@ip-172-31-44-189 ~]$ sudo systemctl start nginx
[ec2-user@ip-172-31-44-189 ~]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-44-189 ~]$ |
```

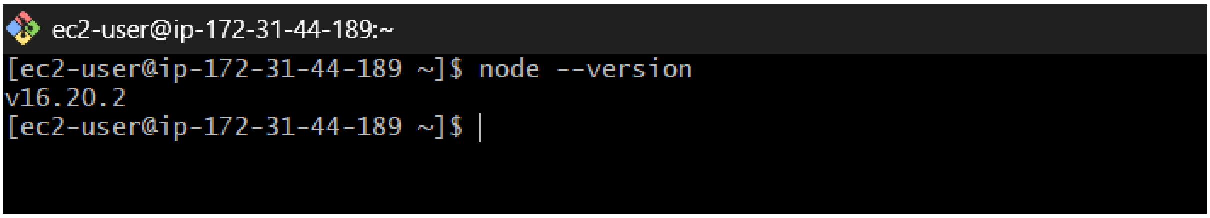
Nginx is a high-performance web server that will serve our React application to users. It efficiently handles HTTP requests and can be configured for load balancing, caching, and serving static files. Enabling the service ensures it starts automatically if the instance restarts.



```
[ec2-user@ip-172-31-44-189 ~]$ sudo service codedeploy-agent status
The AWS CodeDeploy agent is running as PID 13282
[ec2-user@ip-172-31-44-189 ~]$
```



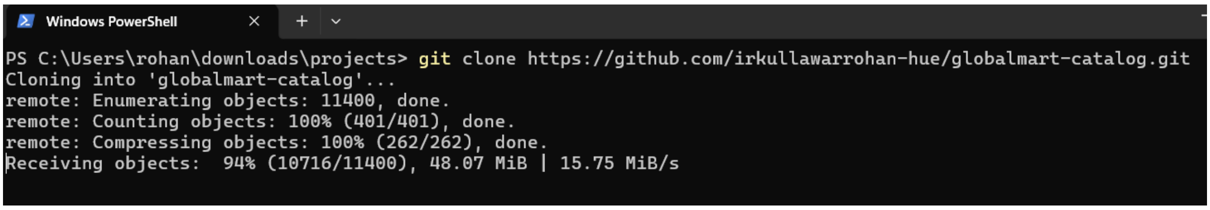
Verified Node Version:



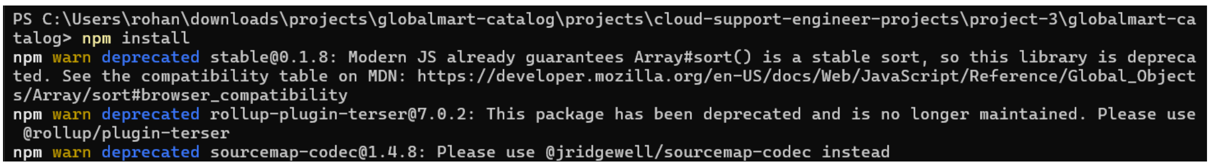
Source Control: GitHub Repository Configuration

Cloned repository to local machine:

Git clone https://github.com/irkullawarrohan-hue/globalmart-catalog.git



Build the React App Locally:



```
PS C:\Users\rohan\downloads\projects\globalmart-catalog\projects\cloud-support-engineer-projects\project-3\globalmart-ca
talog> npm run build

> ecommerce@0.1.0 build
> react-scripts build
```

Building locally allows you to verify the application works as expected before deployment. The build process optimizes the code, minifies files, and creates static assets that are ready for production serving, resulting in better performance for end users.

Modified .gitignore to Include build/ Folder:

```
# dependencies
node_modules
/.pnp
.pnp.js

# testing
/coverage

# production
#/build

# misc
.DS_Store
.env.local
.env.development.local
.env.test.local
.env.production.local

npm-debug.log*
yarn-debug.log*
yarn-error.log*
~
~
```

Then force added the build folder:

git add -f build/

```
PS C:\Users\rohan\downloads\projects\globalmart-catalog\projects\cloud-support-engineer-projects\project-3\globalmart-ca
talog> git add -f build
warning: in the working copy of 'projects/cloud-support-engineer-projects/project-3/globalmart-catalog/build/asset-manif
est.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projects/cloud-support-engineer-projects/project-3/globalmart-catalog/build/static/css/
main.5e67c8af.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projects/cloud-support-engineer-projects/project-3/globalmart-catalog/build/static/js/m
ain.e7776cb7.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projects/cloud-support-engineer-projects/project-3/globalmart-catalog/build/static/js/m
ain.e7776cb7.js.LICENSE.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projects/cloud-support-engineer-projects/project-3/globalmart-catalog/build/static/medi
a/fontawesome-webfont.c1e38fd9e0e74ba58f7a.svg', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\rohan\downloads\projects\globalmart-catalog\projects\cloud-support-engineer-projects\project-3\globalmart-ca
talog>
```

While best practices typically suggest excluding build artifacts from repositories, in this CI/CD implementation we're using a simplified approach where CodeDeploy directly deploys the pre-built files. In more sophisticated pipelines, the build would happen during the CI process instead.

Added deployment configuration files:

Created a file named appspec.yml in the root directory using

This file is required by AWS CodeDeploy to understand how to deploy your application. It defines which files should be copied where and which scripts to run at specific deployment lifecycle events

```yaml
version: 0.0
os: linux
files:
  - source: /build
    destination: /usr/share/nginx/html/
hooks:
  BeforeInstall:
    - location: scripts/before_install.sh
      timeout: 300
      runas: root
  AfterInstall:
    - location: scripts/after_install.sh
      timeout: 300
      runas: root
```

The ***appspec.yml*** configuration instructs CodeDeploy to copy your build folder to Nginx's web directory and run specific scripts before and after installation. This ensures your web server is properly prepared before deployment and correctly configured afterward.

Created deployment scripts

Created scripts directory and two script files:

```bash
#!/bin/bash
# Ensure nginx is installed and deployment directory exists
if [ ! -d /usr/share/nginx/html ]; then
  mkdir -p /usr/share/nginx/html
fi
rm -rf /usr/share/nginx/html/*
~
~
```

Pasted in *scripts/before_install.sh*

The before_install script prepares the deployment environment by ensuring the destination directory exists and clearing any previous files.

```bash
#!/bin/bash
# Configure Nginx for React SPA
cat > /etc/nginx/conf.d/react-app.conf << 'EOL'
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
EOL

# Remove default config if it exists
if [ -f /etc/nginx/conf.d/default.conf ]; then
  rm -f /etc/nginx/conf.d/default.conf
fi

# Set proper permissions and restart nginx
chmod -R 755 /usr/share/nginx/html
systemctl restart nginx
```

Pasted in ***scripts/after_install.sh***

The after_install script configures Nginx toproperly serve your React single-page application

Made scripts executable and commit changes:

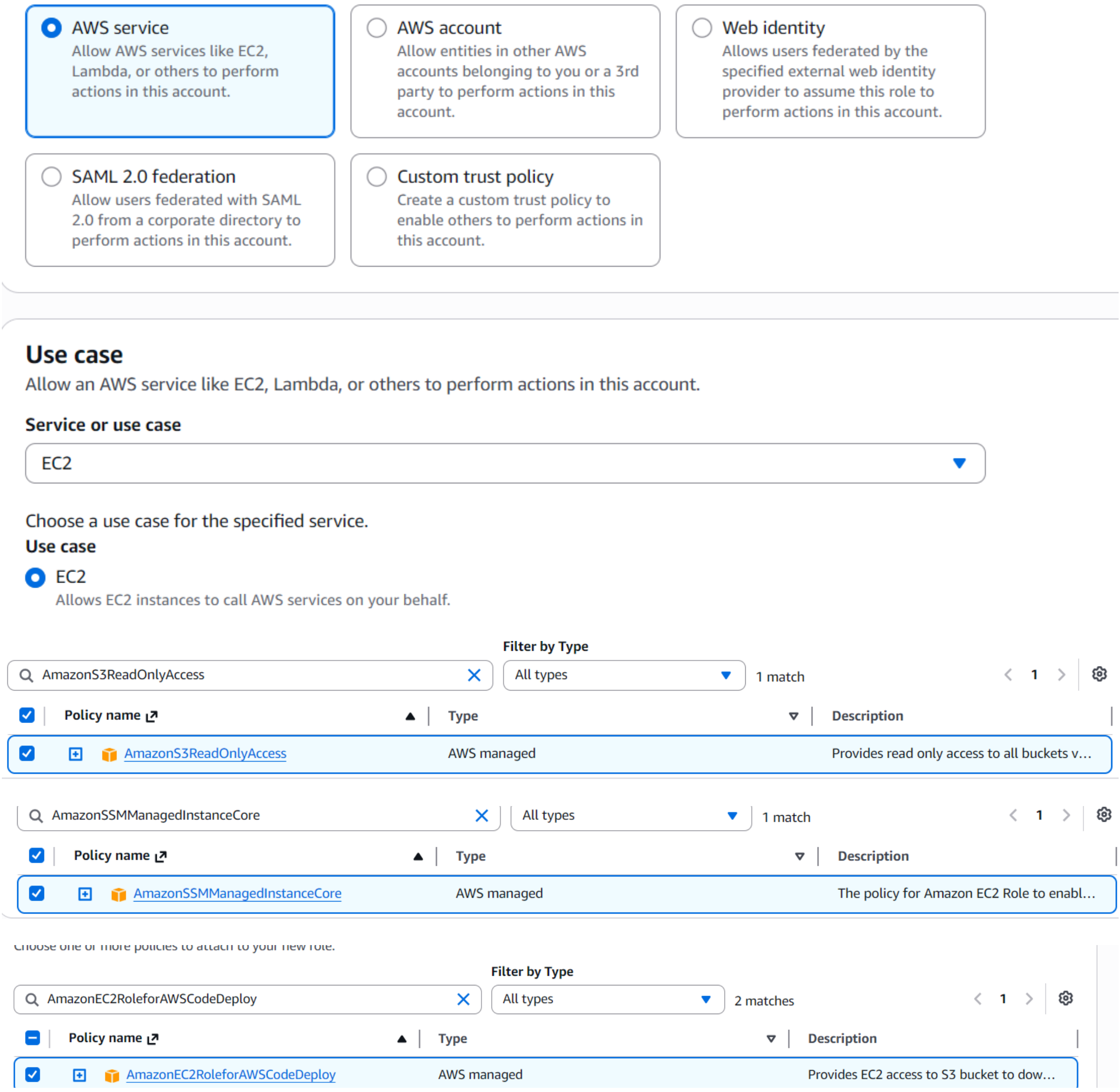
chmod +x scripts/*.sh

git add appspec.yml scripts/

git commit -m "Add deployment configuration for CI/CD pipeline"

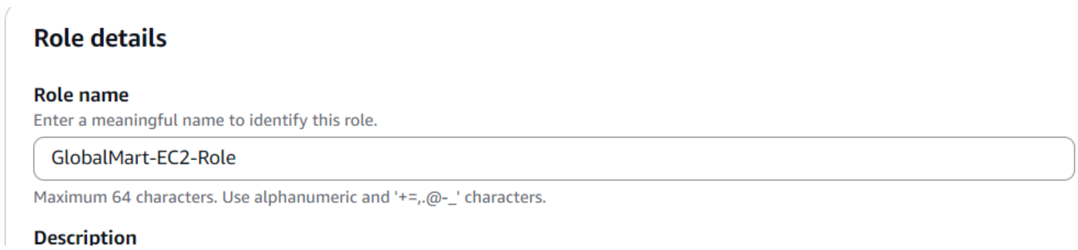
git push origin main

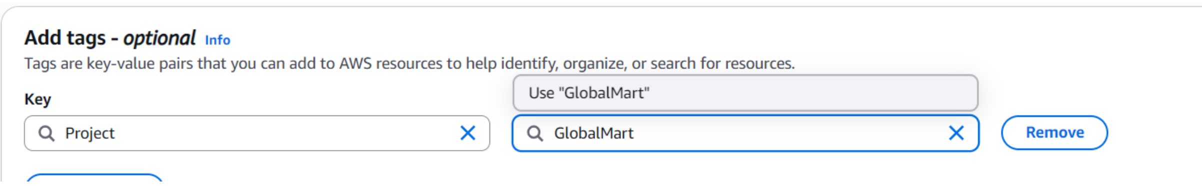
Security: IAM Role Configuration



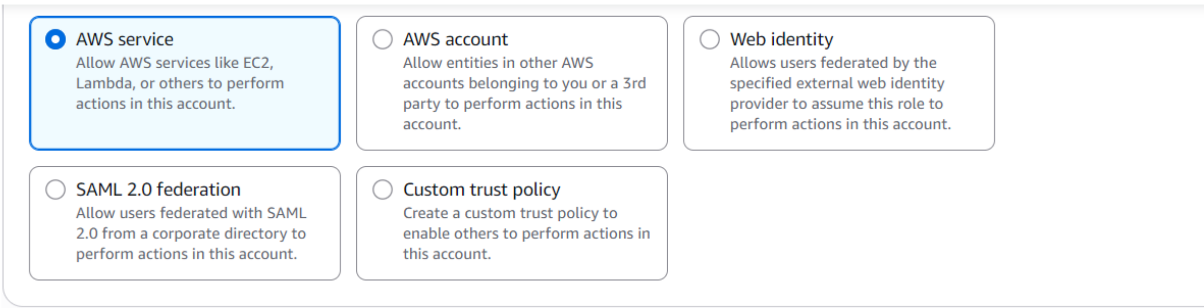
This combination allows your EC2 instance to:

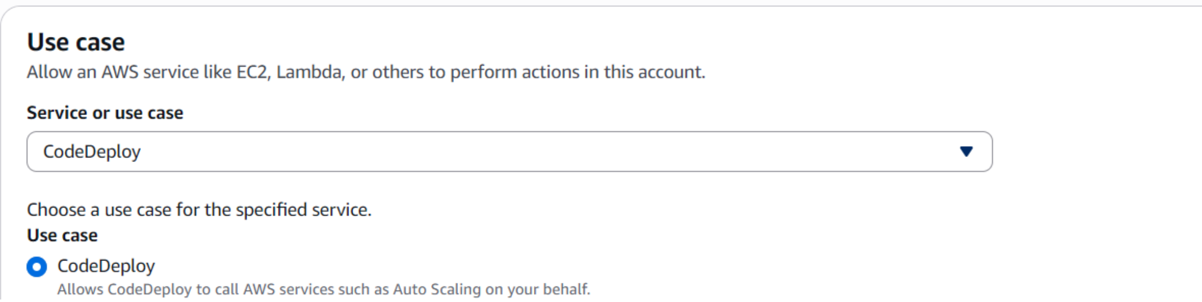
- Access deployment artifacts from S3 buckets (S3ReadOnlyAccess)
- Be managed remotely via AWS Systems Manager (SSMManagedInstanceCore)
- Interact with CodeDeploy service to receive deployments (EC2RoleforAWSCodeDeploy)

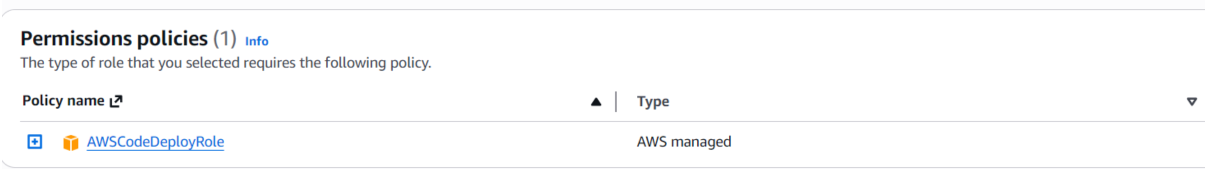


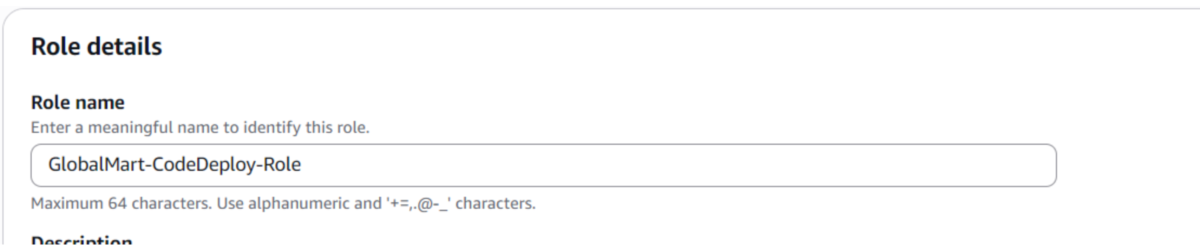


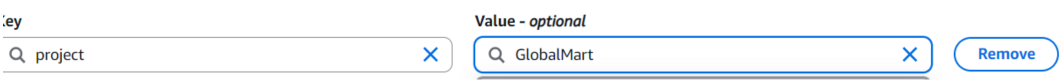
Created CodeDeploy Role:



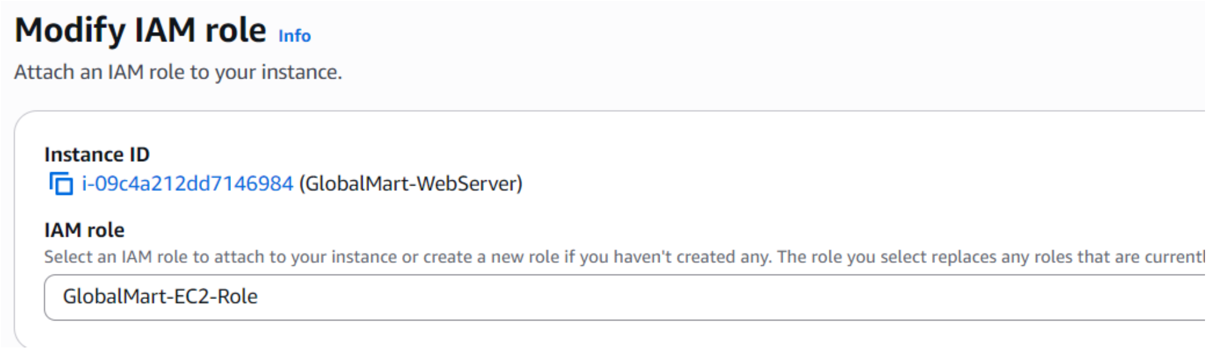






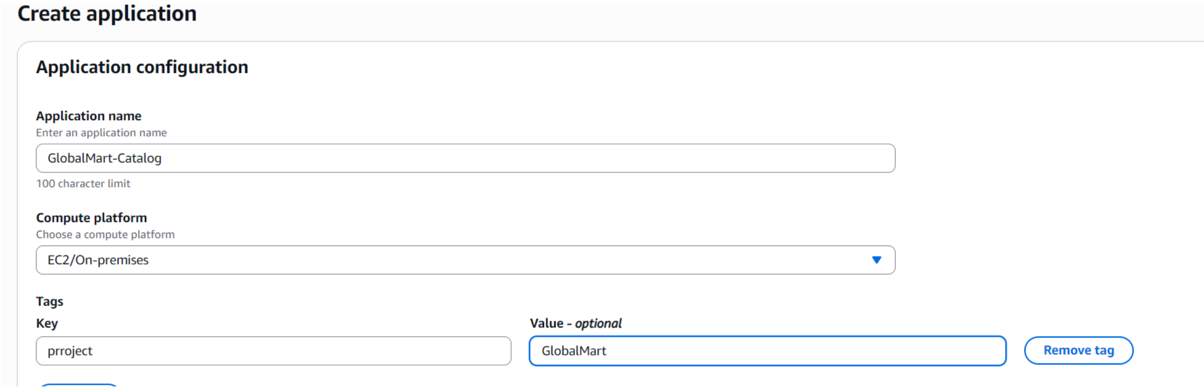


Attached EC2 Role to your instance

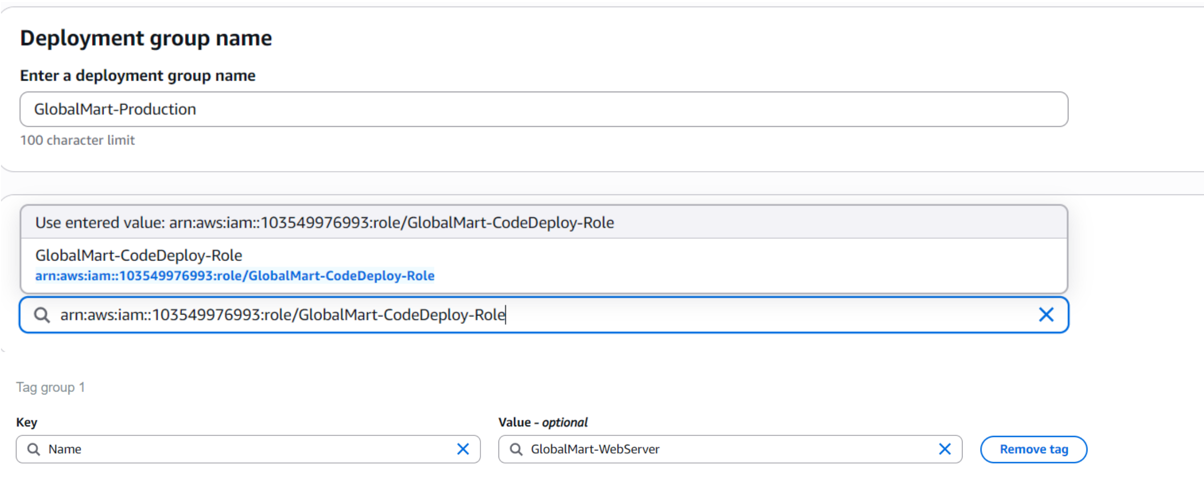


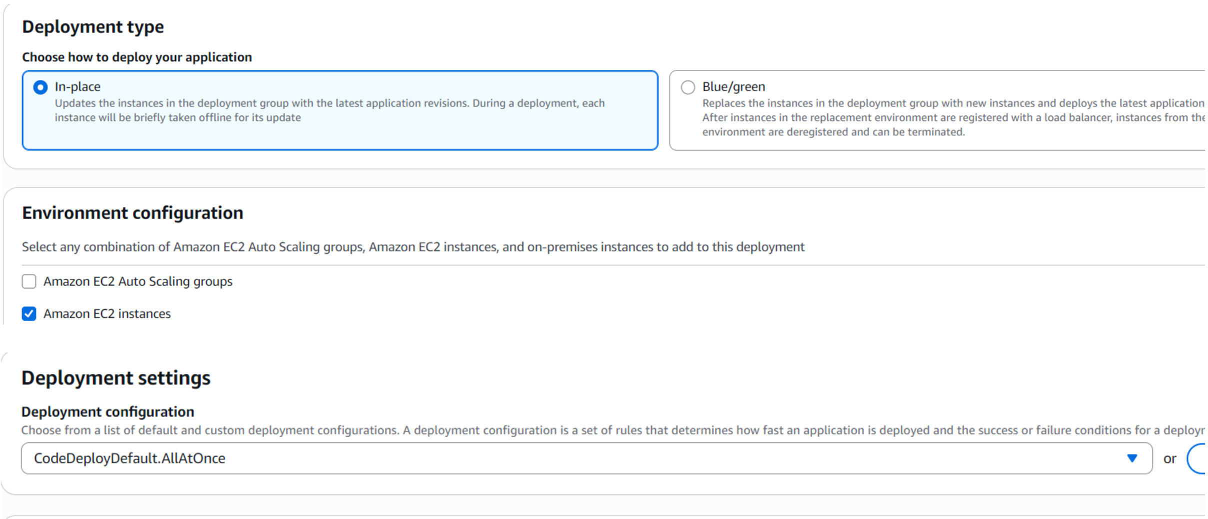
Deployment: AWS CodeDeploy Setup

Created CodeDeploy Application:



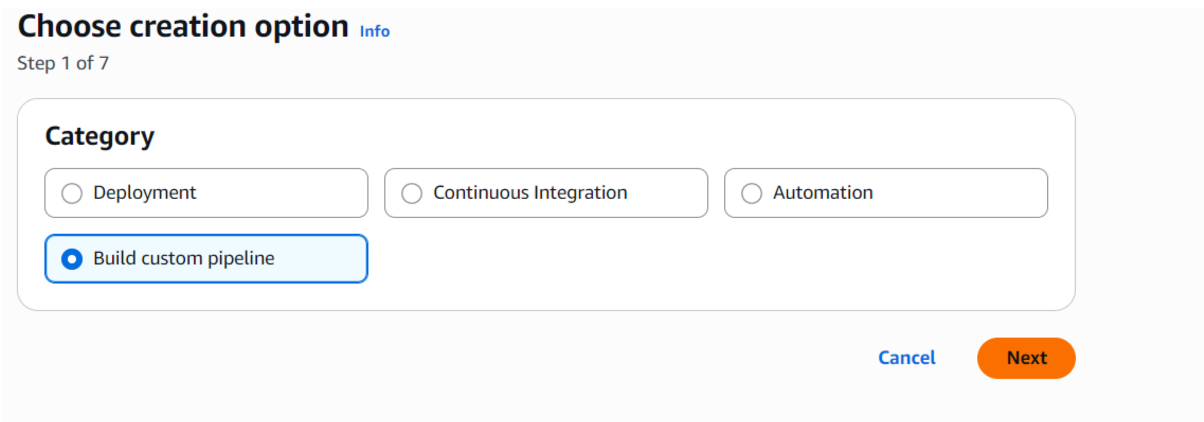
Created Deployment Group

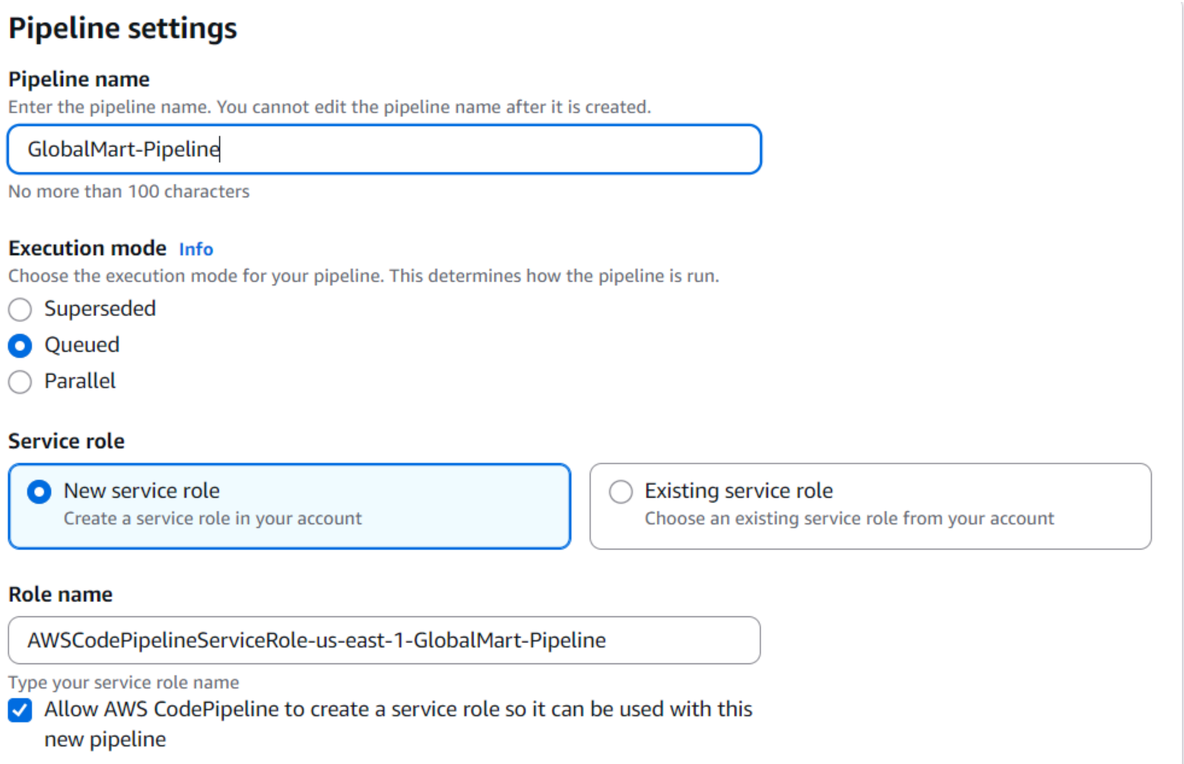




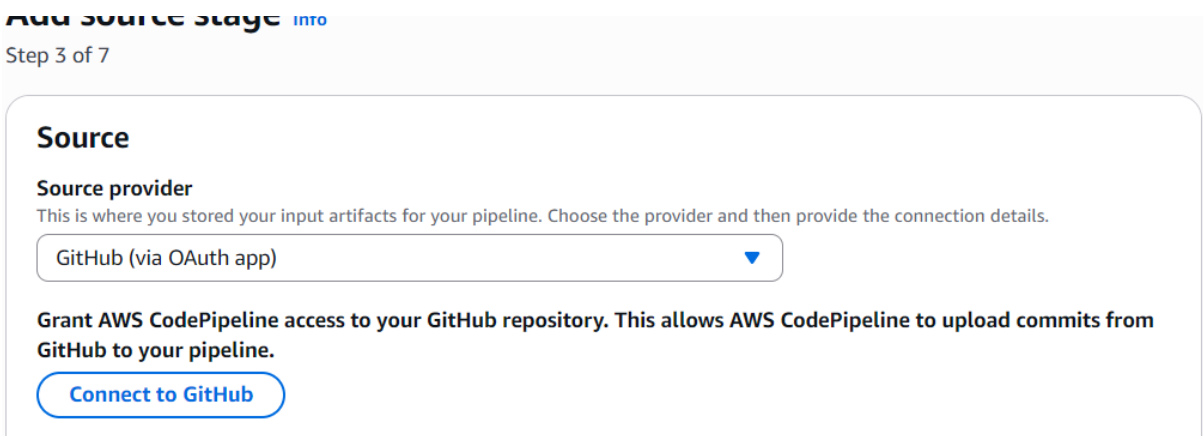
Pipeline: AWS CodePipeline Configuration

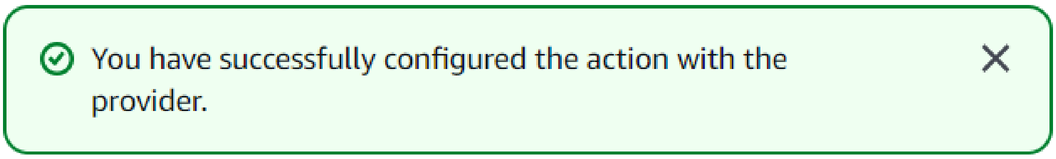
Created Pipeline

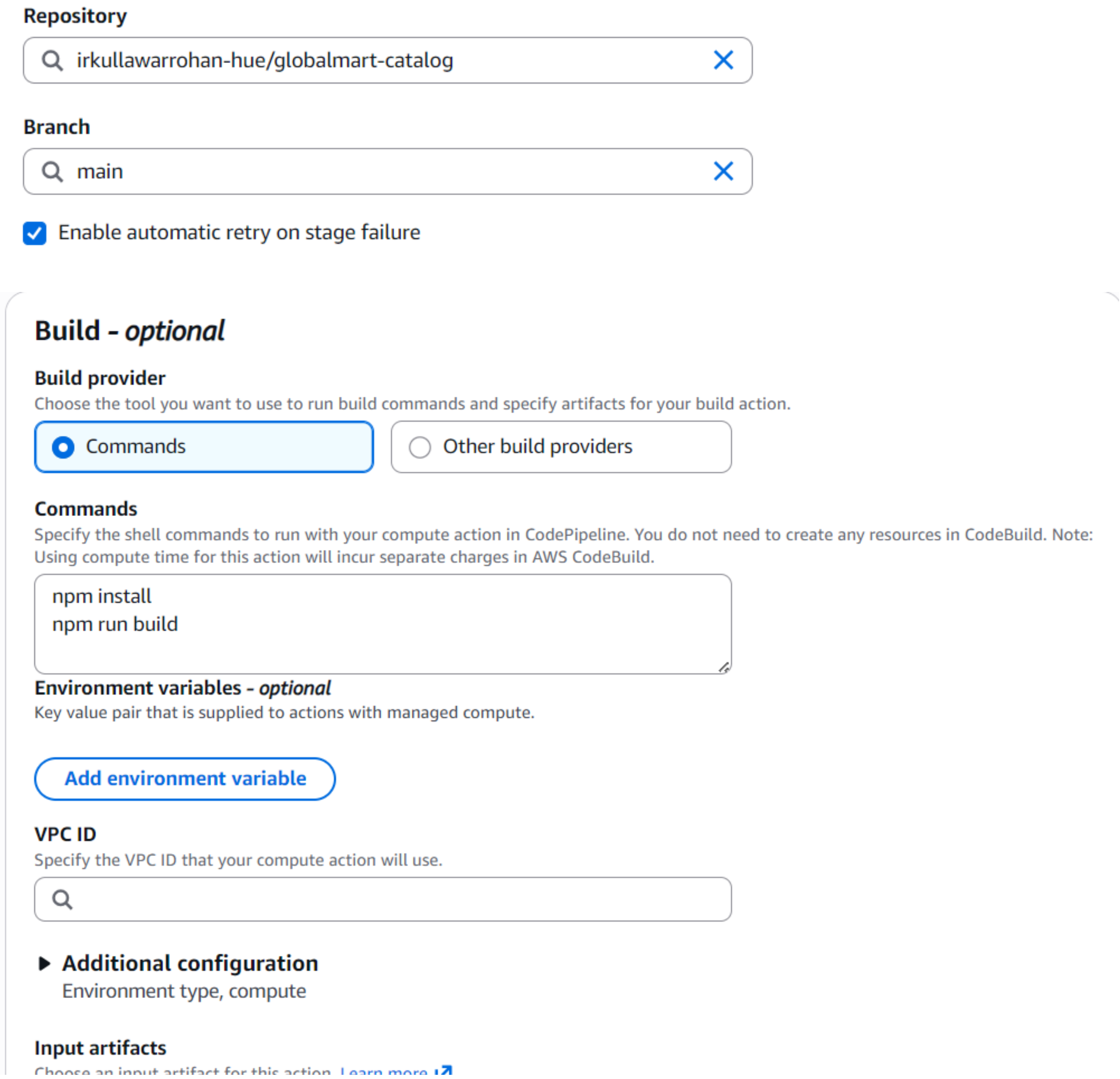


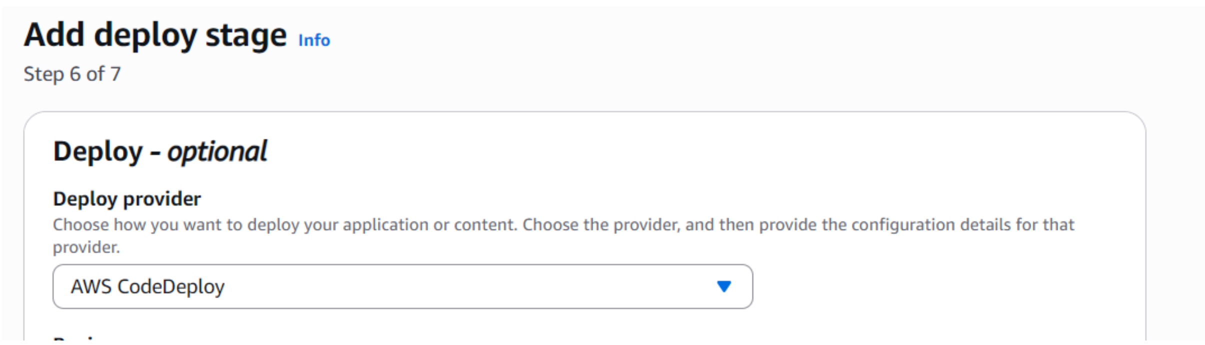


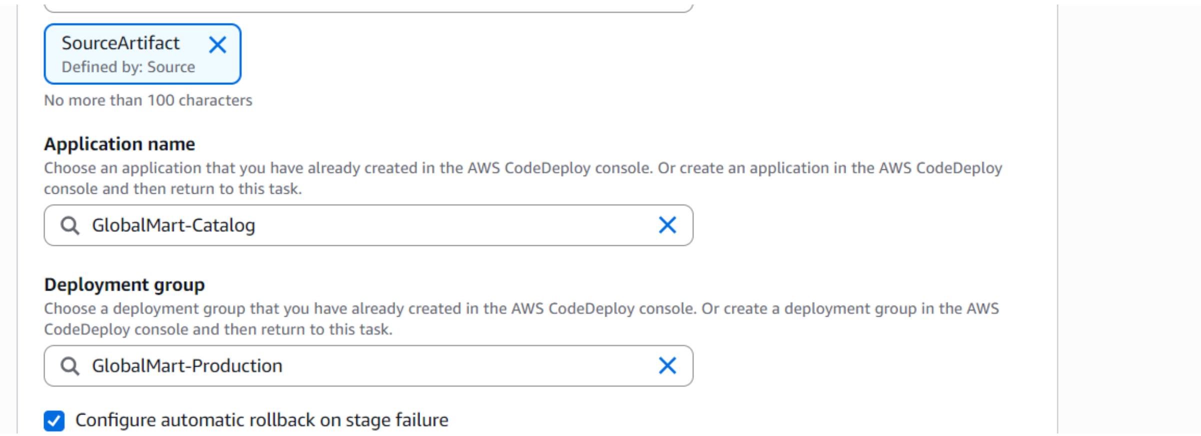
The queued execution mode ensures that only one pipeline execution runs at a time, preventing conflicts when multiple code changes are pushed rapidly.



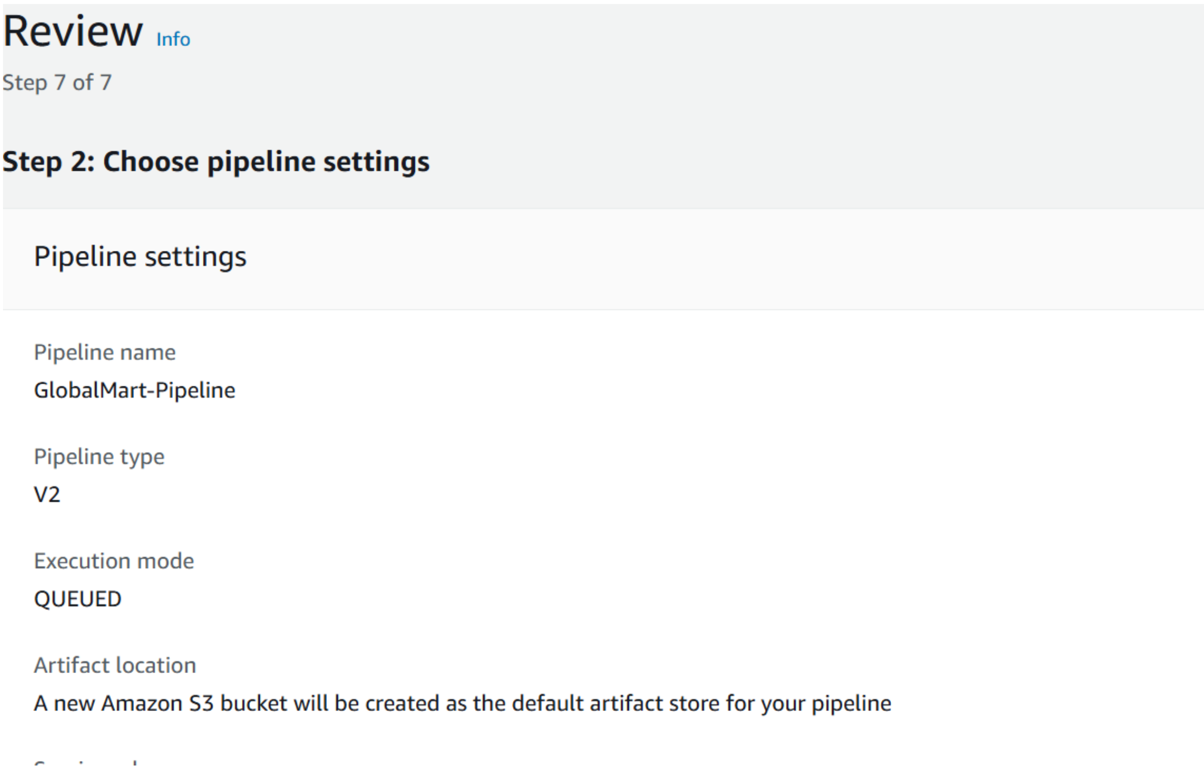



Repository
irkullawarrohan-hue/globalmart-catalog
Branch
main
Enable automatic retry on stage failure
Build - optional
Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.
Commands
Other build providers
Commands
Specify the shell commands to run with your compute action in CodePipeline. You do not need to create any resources in CodeBuild. Note: Using compute time for this action will incur separate charges in AWS CodeBuild.
npm install
npm run build
Environment variables - optional
Key value pair that is supplied to actions with managed compute.
Add environment variable
VPC ID
Specify the VPC ID that your compute action will use.
Additional configuration
Environment type, compute
Input artifacts


Add deploy stage
Info
Step 6 of 7
Deploy - optional
Deploy provider
Choose how you want to deploy your application or content. Choose the provider, and then provide the configuration details for that provider.
AWS CodeDeploy



Reviewed and created pipeline:



Step 3: Add source stage

Source action provider

Source action provider
GitHub (via OAuth app)

PollForSourceChanges
false

Repo
globalmart-catalog

Owner
Snigdha-Chaudhari

Branch
main

Enable automatic retry on stage failure

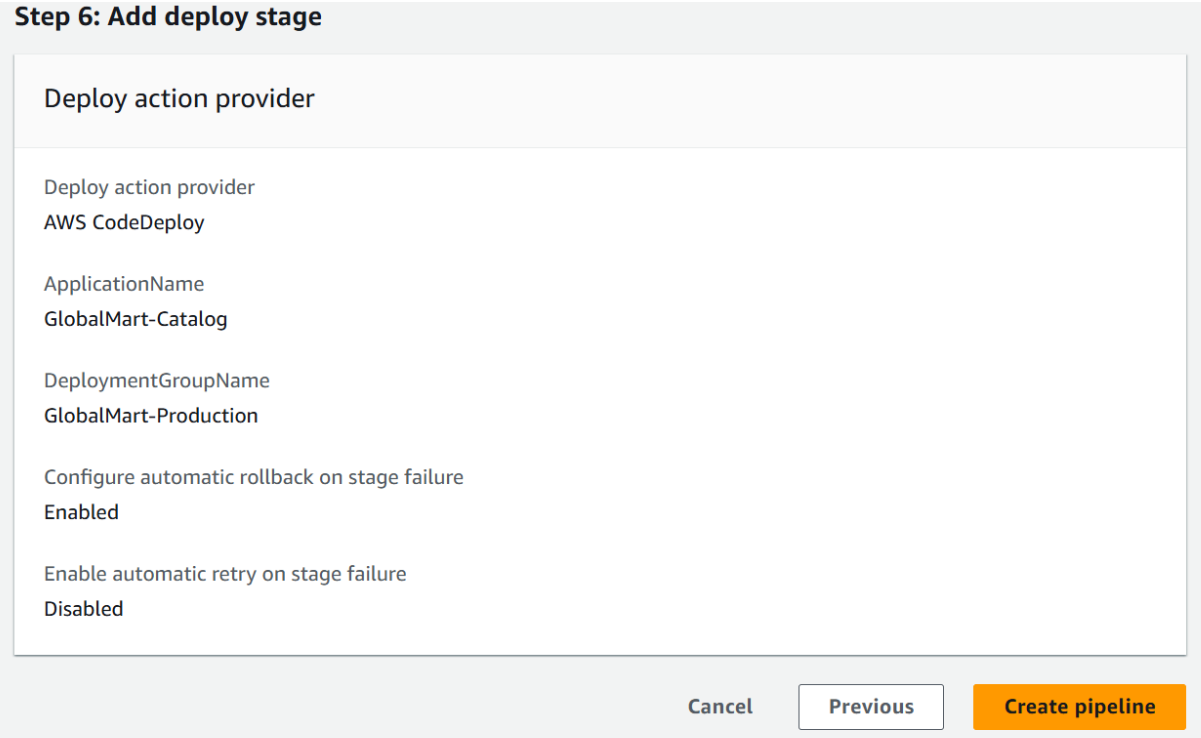
Step 4: Add build stage

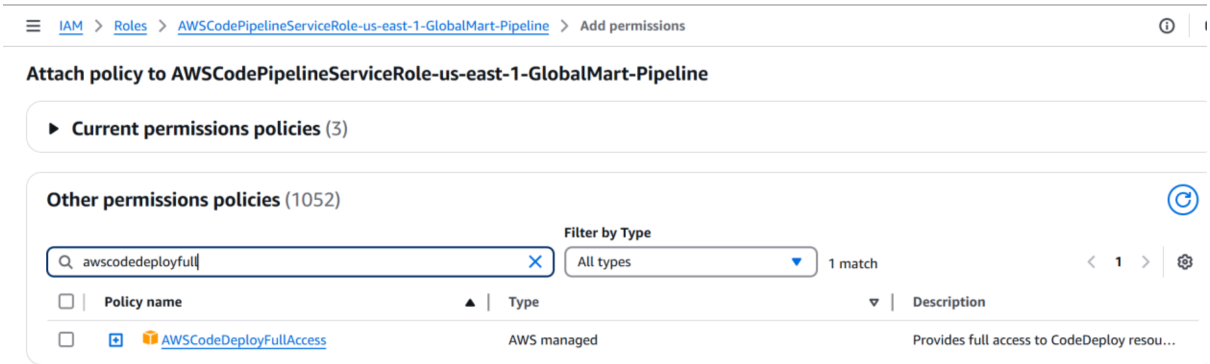
Build action provider

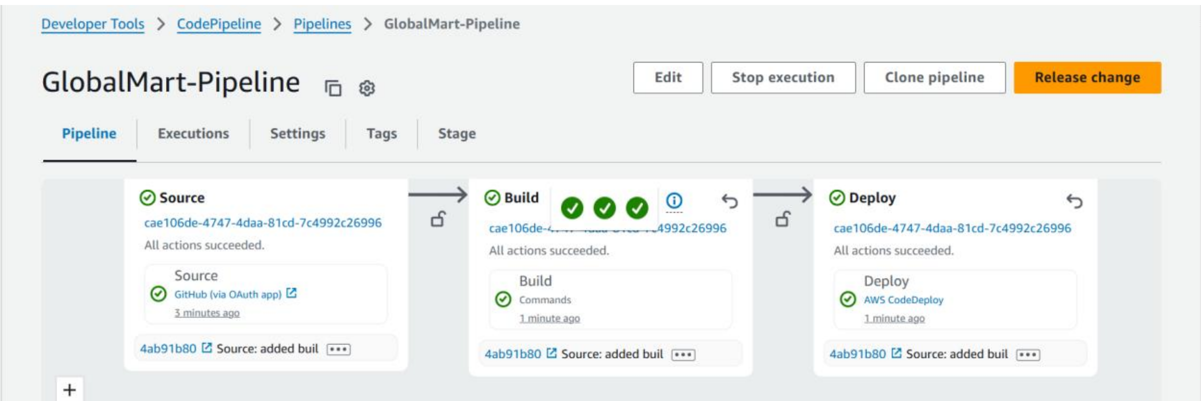
Build action provider
Commands

Commands
["version: 0.2","phases:"," install:"," runtime-versions:"," nodejs: 16"," pre_build:"," commands:"," - npm install"," build:"," commands:"," - npm run build","artifacts:"," files:"," - build/**/*"," - appspec.yml"," - scripts/**/*"]

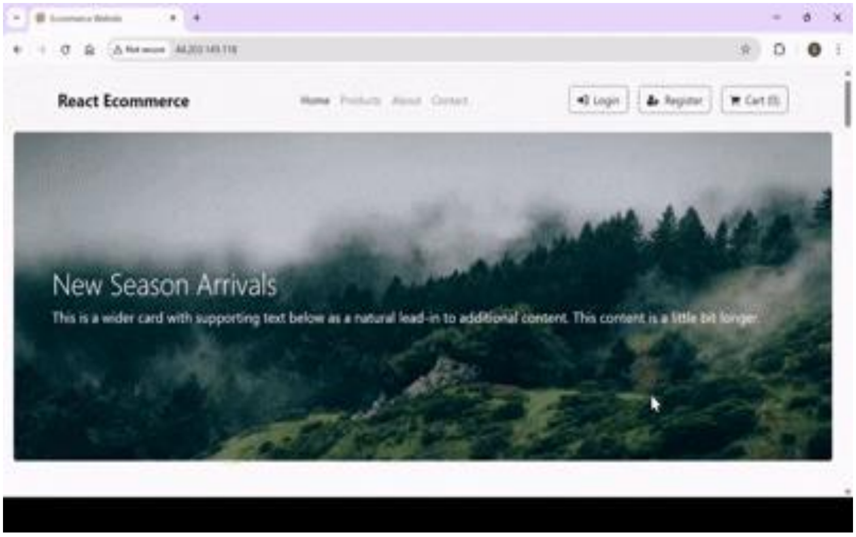
Enable automatic retry on stage failure
Enabled


Step 6: Add deploy stage
Deploy action provider
Deploy action provider
AWS CodeDeploy
ApplicationName
GlobalMart-Catalog
DeploymentGroupName
GlobalMart-Production
Configure automatic rollback on stage failure
Enabled
Enable automatic retry on stage failure
Disabled
Cancel
Previous
Create pipeline


IAM > Roles > AWSCodePipelineServiceRole-us-east-1-GlobalMart-Pipeline > Add permissions
Attach policy to AWSCodePipelineServiceRole-us-east-1-GlobalMart-Pipeline
Current permissions policies (3)
Other permissions policies (1052)
Filter by Type
awscodedeployfull
All types
1 match
1
Policy name
Type
Description
AWSCodeDeployFullAccess
AWS managed
Provides full access to CodeDeploy resou…


Developer Tools > CodePipeline > Pipelines > GlobalMart-Pipeline
GlobalMart-Pipeline
Edit
Stop execution
Clone pipeline
Release change
Pipeline
Executions
Settings
Tags
Stage
Source
cae106de-4747-4daa-81cd-7c4992c26996
All actions succeeded.
Source
GitHub (via OAuth app)
3 minutes ago
4ab91b80
Source: added buil
Build
All actions succeeded.
Build
Commands
1 minute ago
4ab91b80
Source: added buil
Deploy
cae106de-4747-4daa-81cd-7c4992c26996
All actions succeeded.
Deploy
AWS CodeDeploy
1 minute ago
4ab91b80
Source: added buil

Verified deployment:



CI/CD Pipeline Troubleshooting

Introduction

Now that I built a working CI/CD pipeline, this section covers the common issues I encountered and worked through — just as a DevOps engineer would in real-world scenarios. By intentionally "breaking" components of the pipeline and then resolving them, I gained valuable troubleshooting experience.

Important Note: Before proceeding, I took notes about my working configuration and created screenshots of key settings. This helped me remember the correct configuration while troubleshooting.

Methodical Troubleshooting Approach When troubleshooting CI/CD pipeline issues, I followed these general steps:

1. Identified the failure point in the pipeline
2. Checked the logs for specific error messages
3. Verified configurations of the relevant AWS services
4. Tested my fix to ensure it resolved the issue
5. Documented the solution for future reference

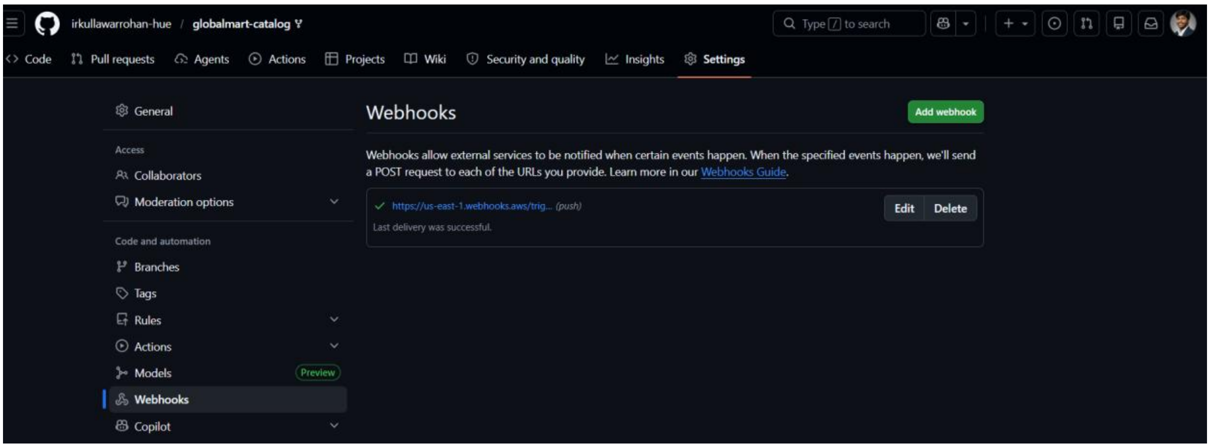
Issue 1: GitHub Webhook Connection

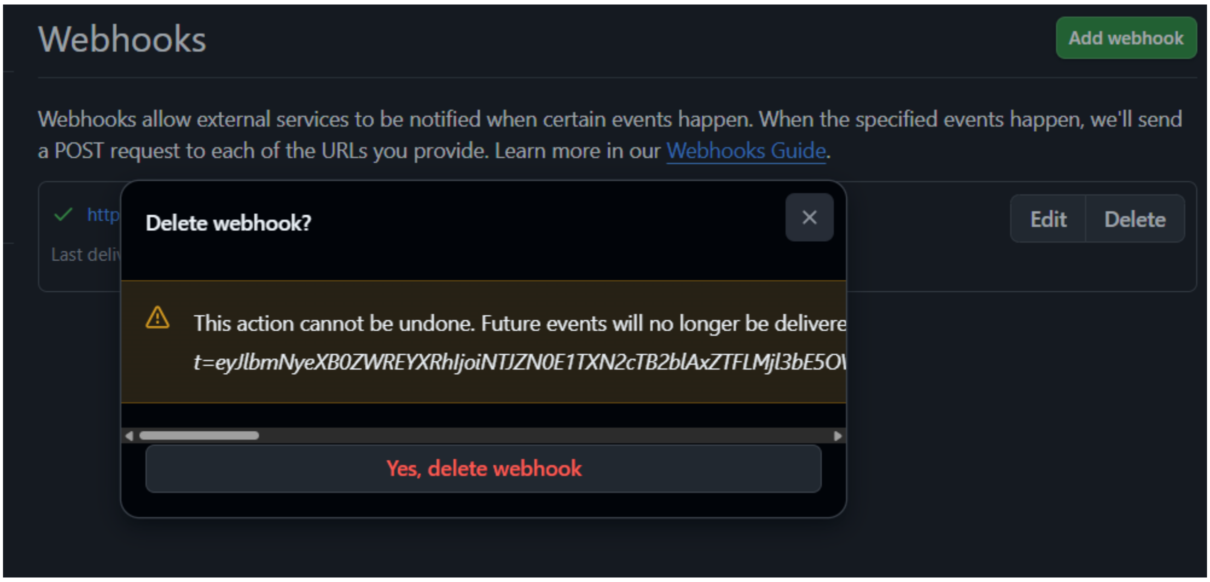
Breaking the Pipeline:

I navigated to my GitHub repository -

- Went to my forked repository on GitHub
- Clicked on "Settings" tab
- Selected "Webhooks" from the left sidebar
- Found the webhook created by AWS CodePipeline (it had an AWS URL)
- Clicked on the webhook, scrolled down and clicked "Delete"
- Confirmed deletion

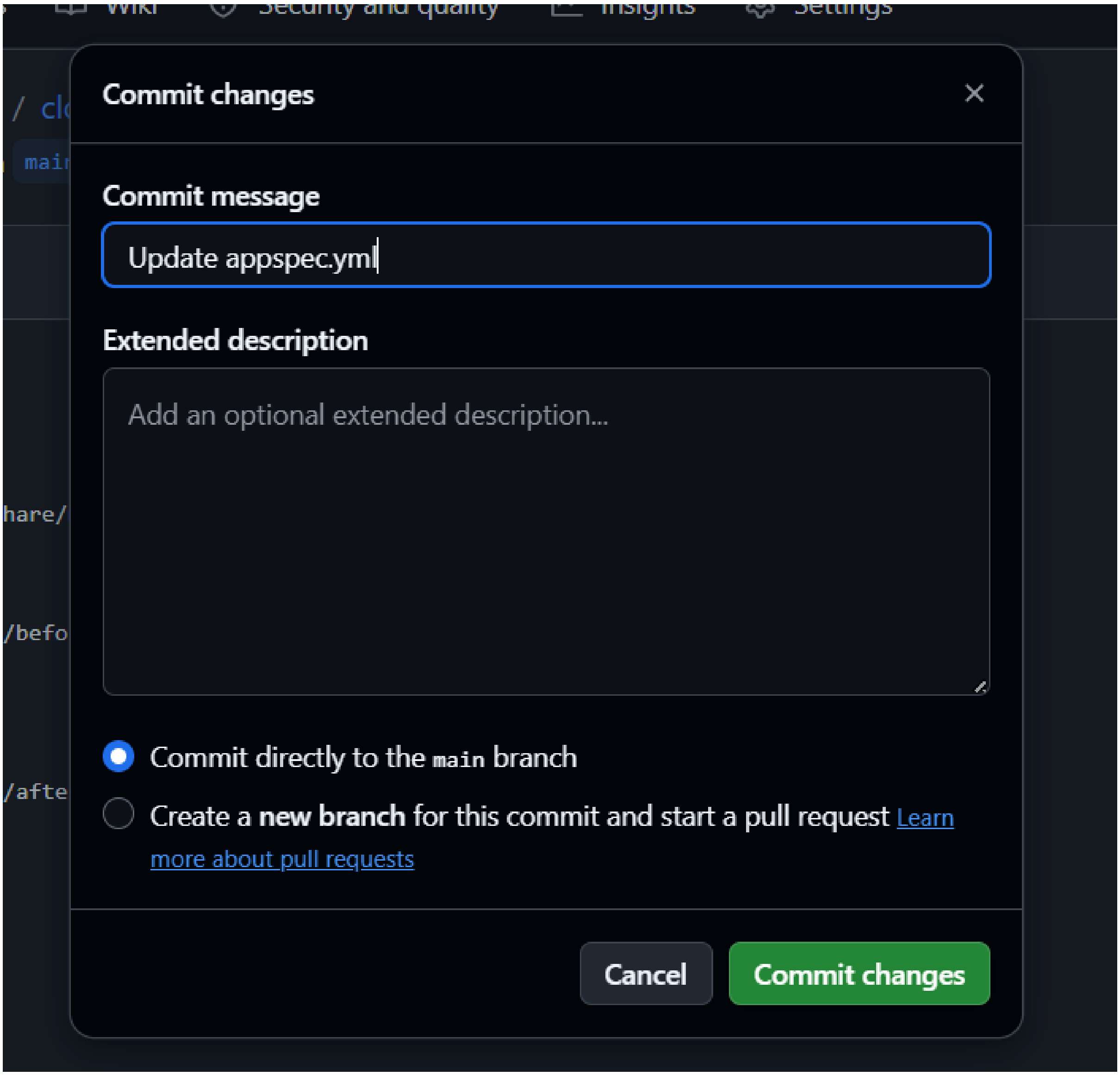
By deleting this webhook, I broke the automatic communication link between my code repository and the deployment pipeline.





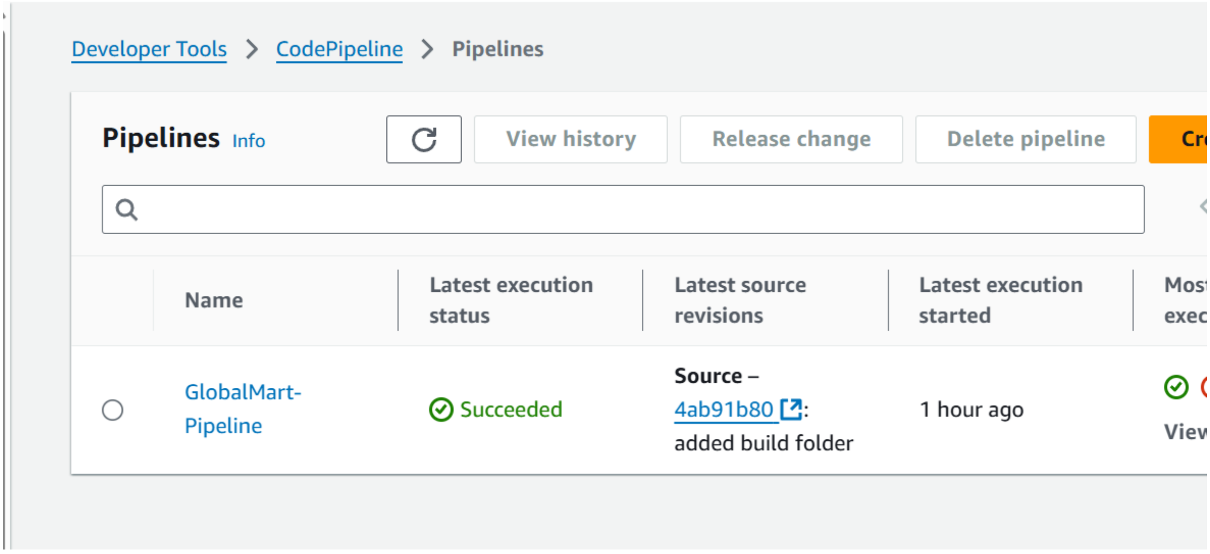
I tested the broken webhook:

- Made a small change to a file in my repository (changed version to 0.1 from 0.0 in appspec.yml)
- Committed and pushed the change
- Went to AWS CodePipeline and observed that the pipeline did not automatically start



Symptoms:

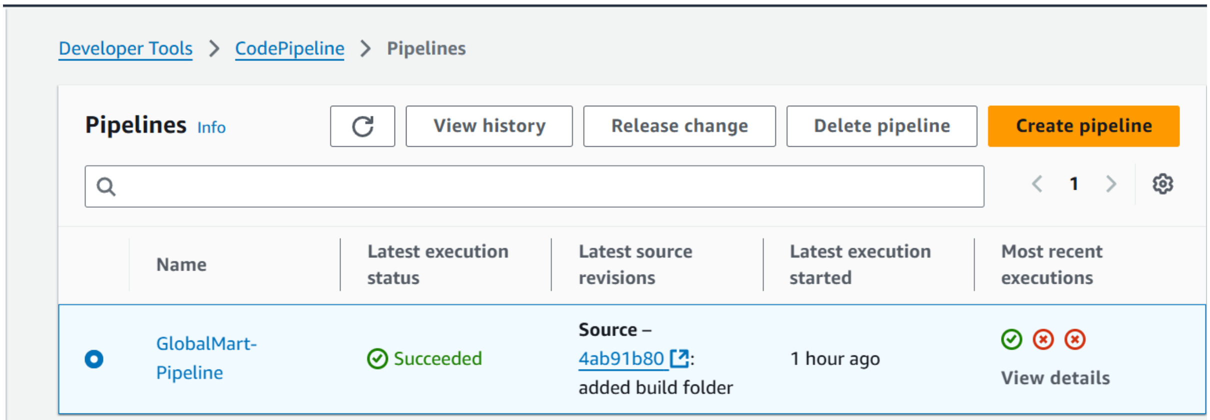
- Changes pushed to GitHub didn't trigger the pipeline
- No new executions appeared in CodePipeline history

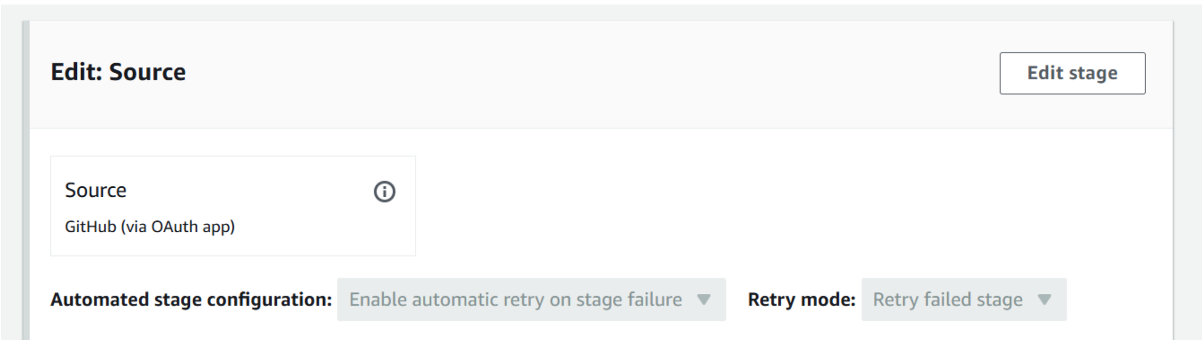


Diagnosis:

I checked pipeline configuration:

- Went to CodePipeline > my pipeline
- Clicked "Edit" and reviewed the Source stage settings



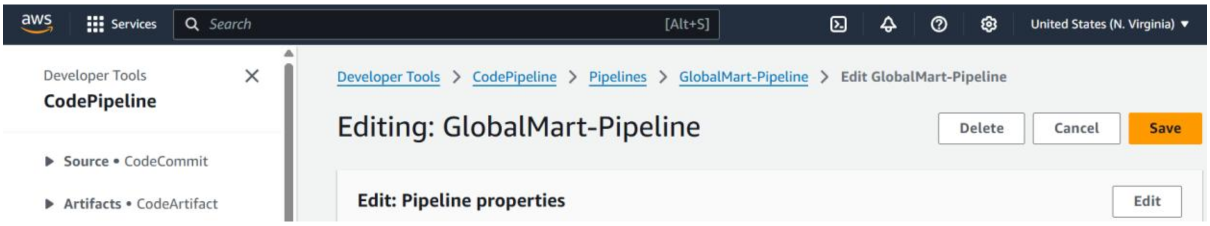


I checked GitHub repository settings:

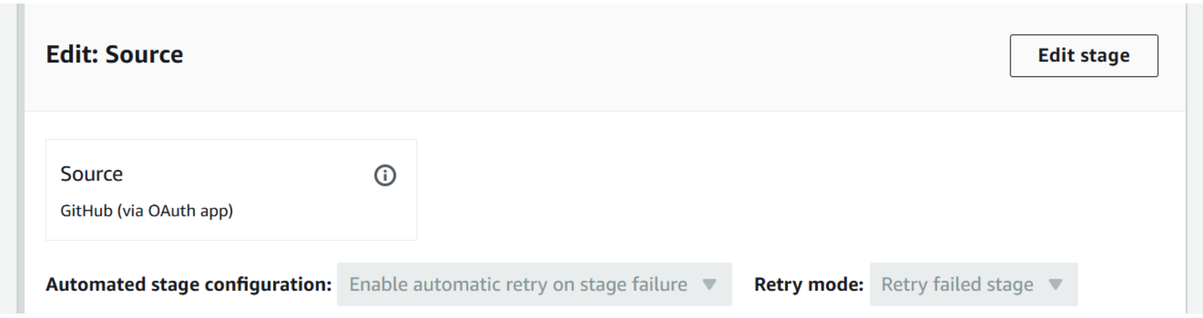
- Went to repository > Settings > Webhooks
- Confirmed no AWS webhook existed — this confirmed the webhook issue

Solution:

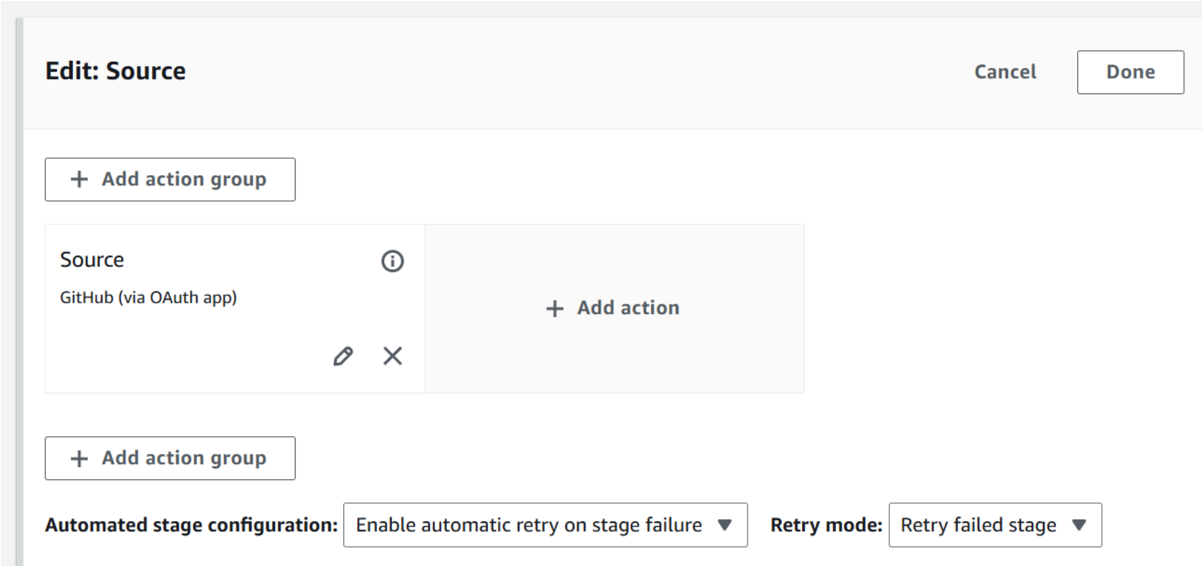
Went to CodePipeline > GlobalMart-Pipeline > Edit



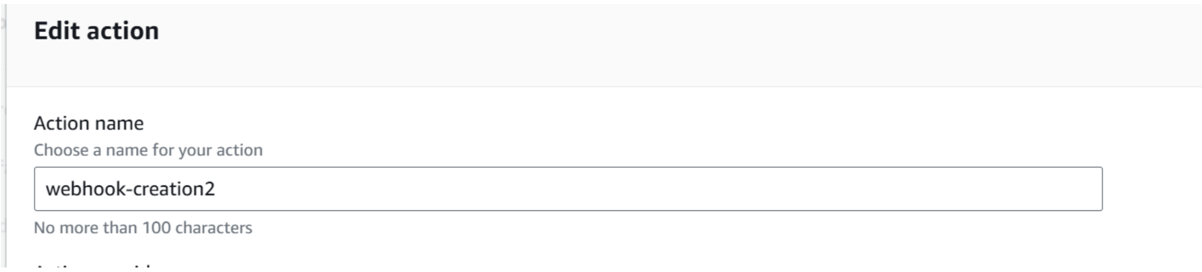
Clicked "Edit" on the Source stage



Clicked "Add action"

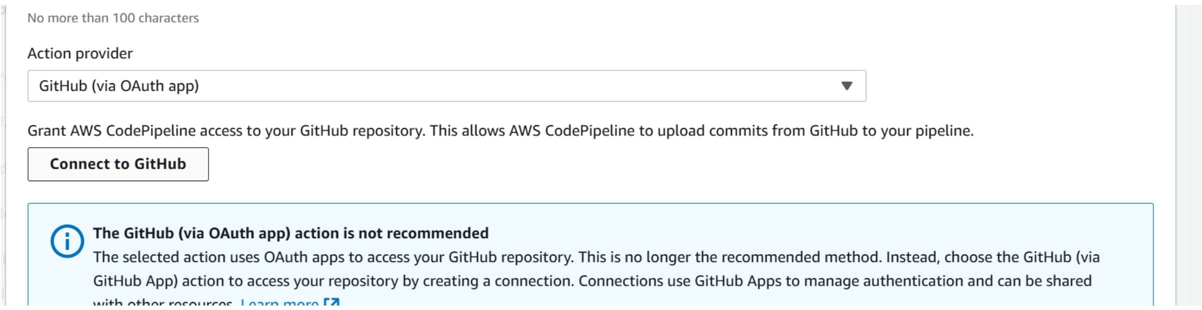


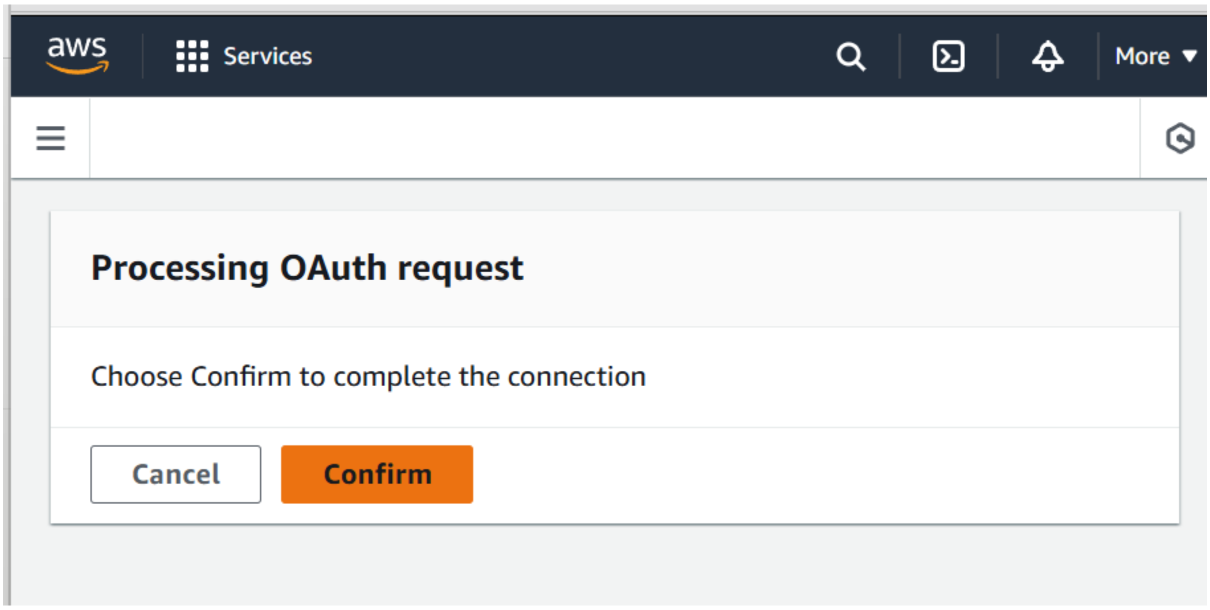
Provided Action name: webhook-creation2



Selected Action provider: "GitHub (via OAuth app)"

Clicked "Connect to GitHub" > "Confirm"





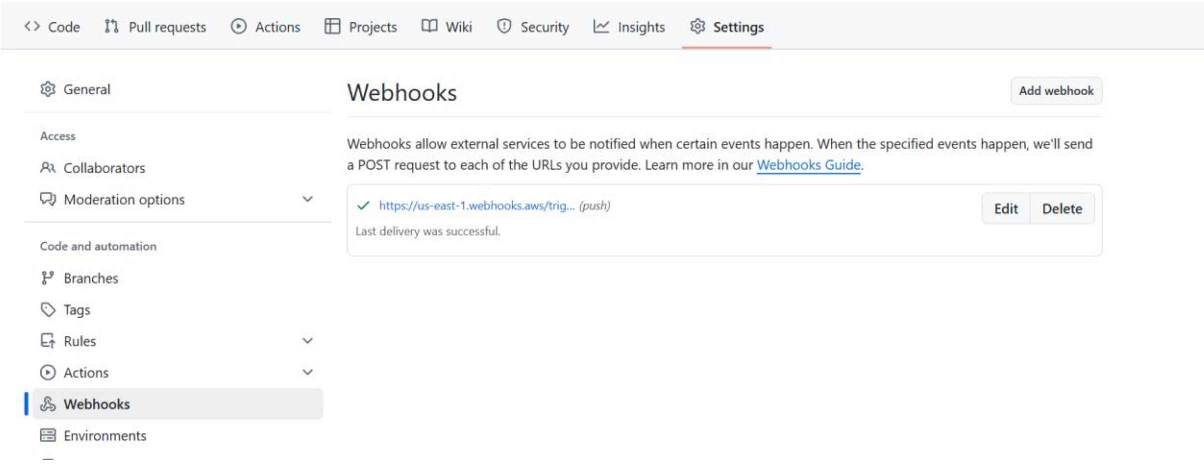
Selected my repository and branch:

Provided artifact name: webhook-c2

Clicked "Done" then "Save" — this recreated the webhook automatically

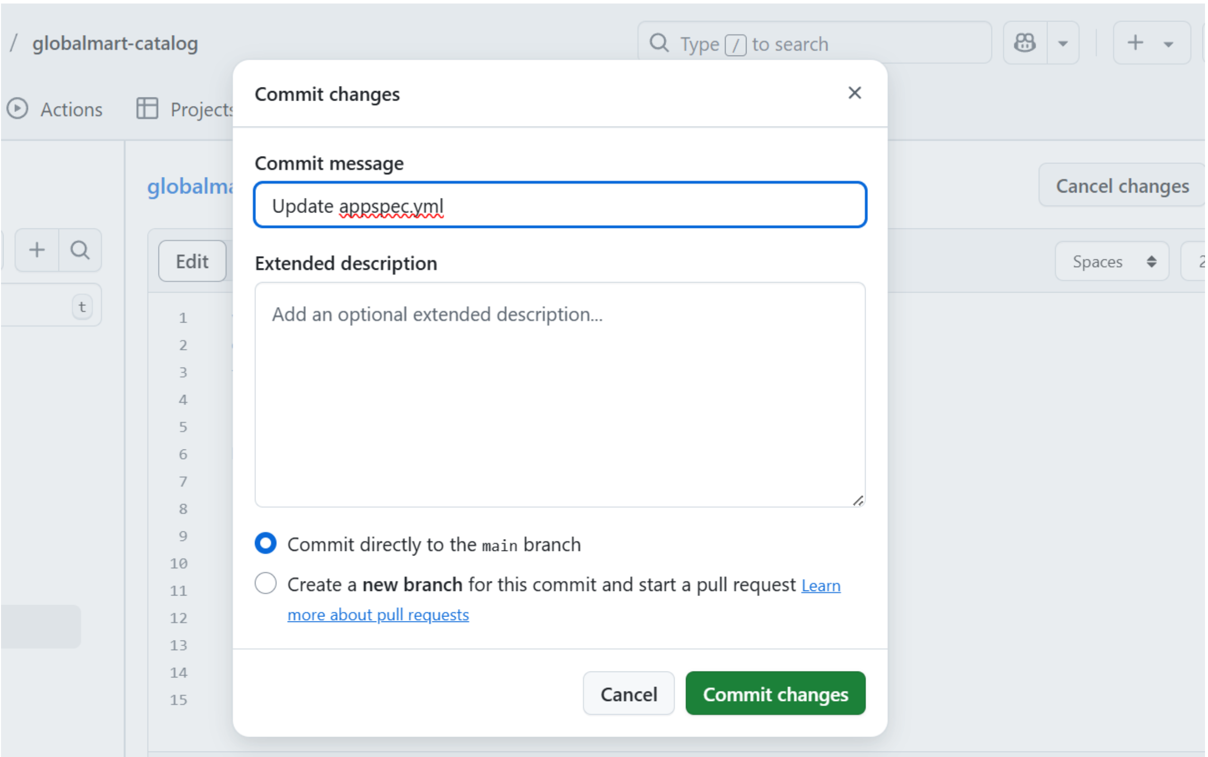
I verified webhook creation:

- Went to GitHub repository > Settings > Webhooks
- Confirmed a new webhook from AWS was created

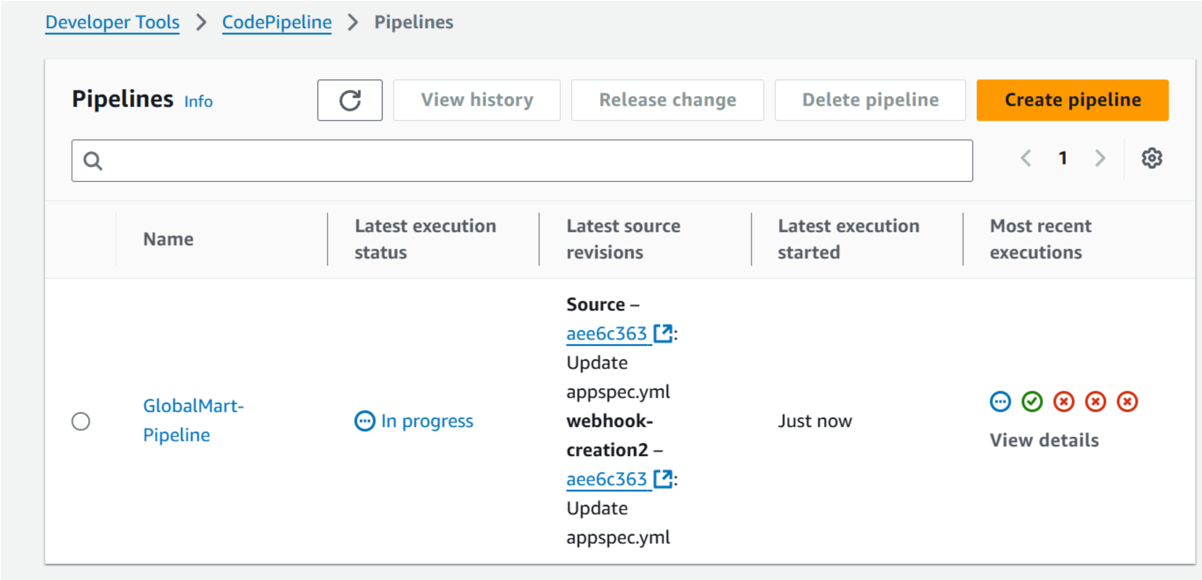


I tested the fix:

- Made a small change, committed and pushed
- Verified the pipeline triggered automatically



Verified the pipeline triggers automatically:

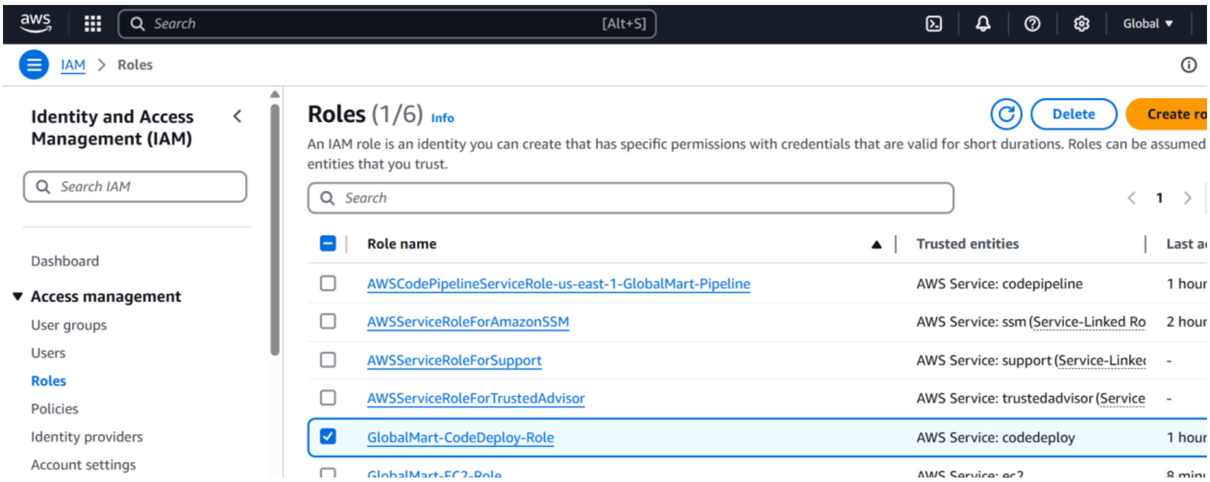


Issue 2: IAM Permissions Problem

I navigated to IAM in the AWS Console:

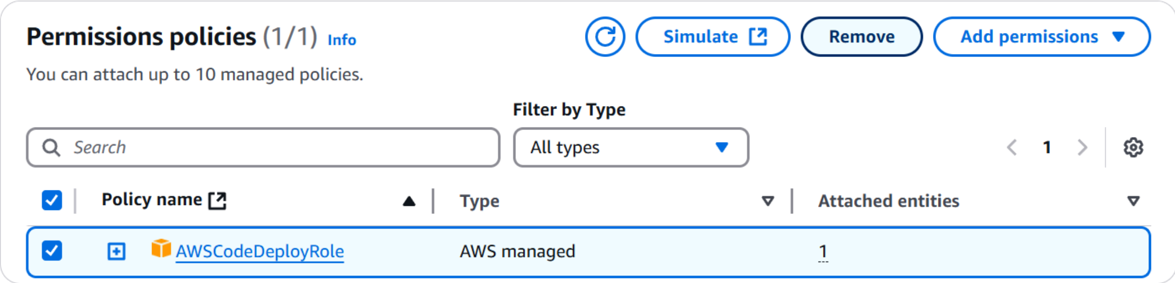
-Went to IAM > Roles

-Searched for and selected GlobalMart-CodeDeploy-Role

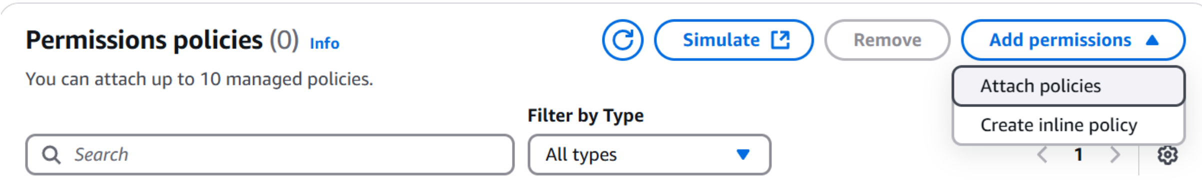


- Went to the "Permissions" tab

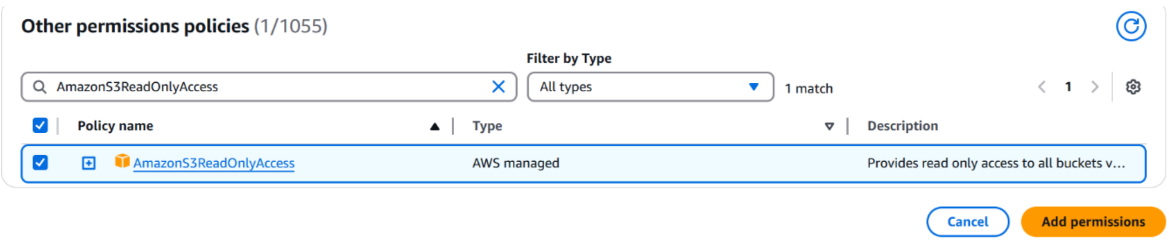
- Removed the AWSCodeDeployRole policy



Clicked "Add permissions" > "Attach policies"

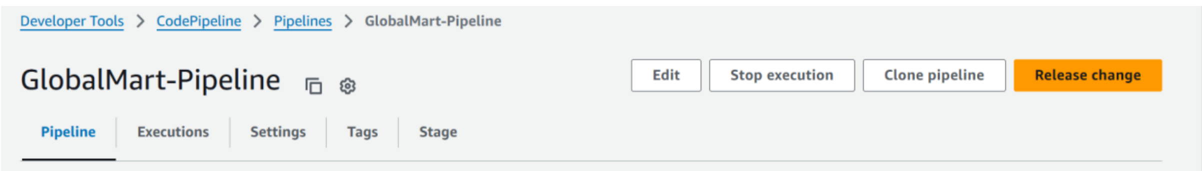


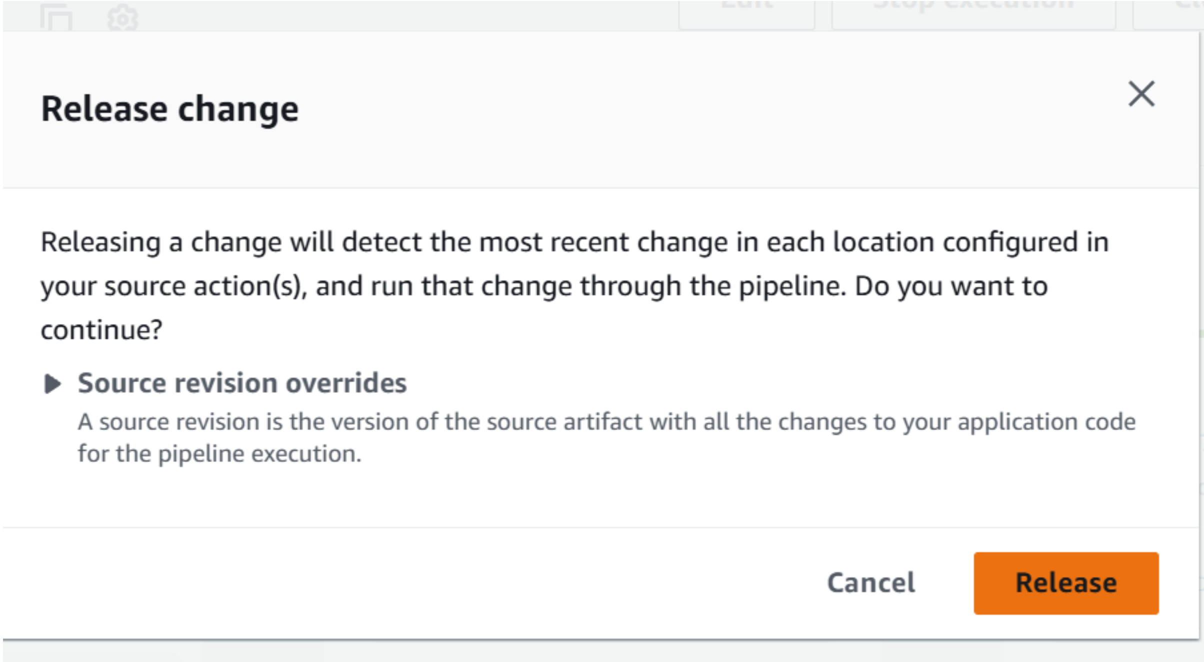
Searched for and attached AmazonS3ReadOnlyAc



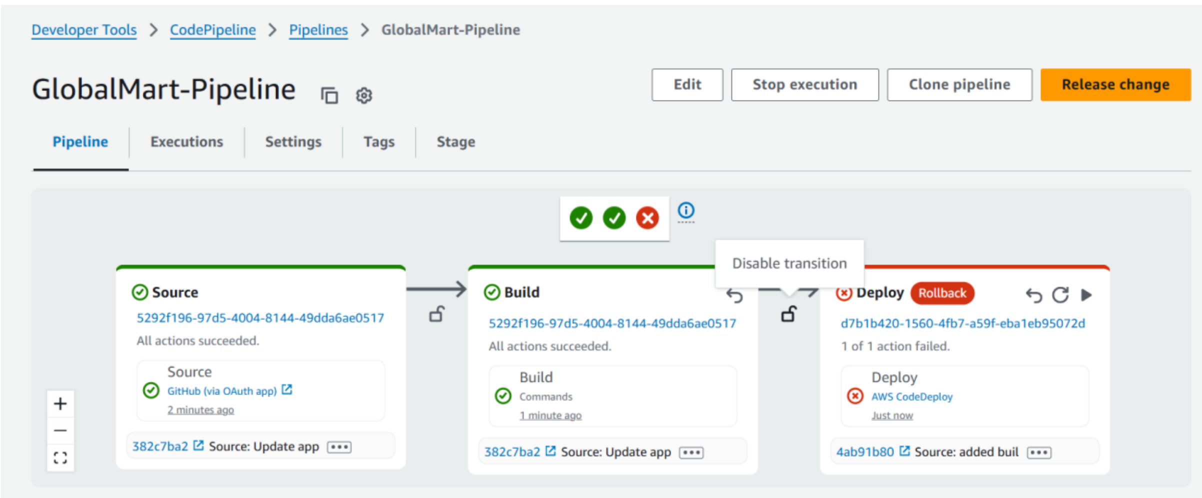
This severely restricted CodeDeploy to only reading from S3, preventing it from performing most of its core functions.

I tested the permission issue:





Observed that the pipeline failed during the Deploy stage



Reviewed the error messages in CodeDeploy

When a pipeline fails specifically at the Deploy stage (after successfully completing Source and Build stages), it suggests that the issue lies with the deployment service rather than with the code itself. Permission-related error messages in CodeDeploy logs further narrow down the problem to IAM configuration rather than application or network issues.

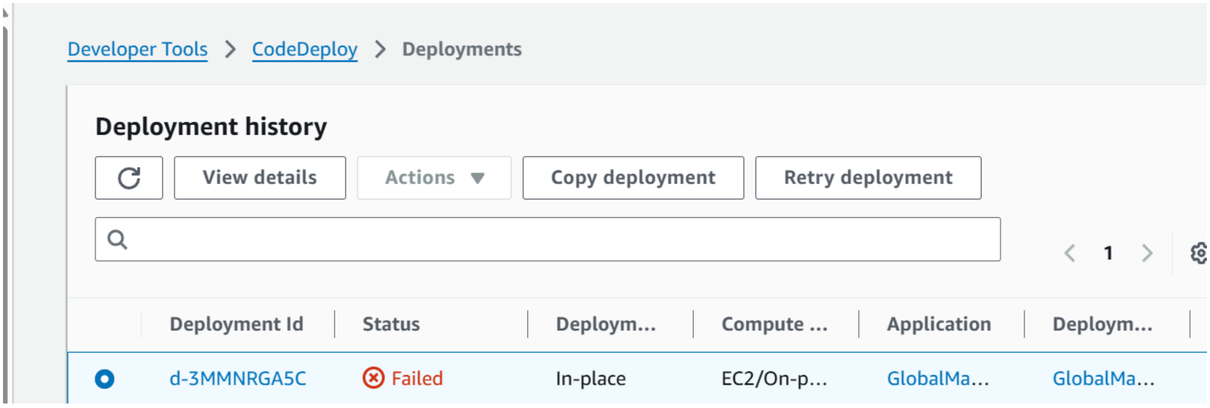
Symptoms:

- Pipeline fails at the Deploy stage
- CodeDeploy shows permission-related error messages

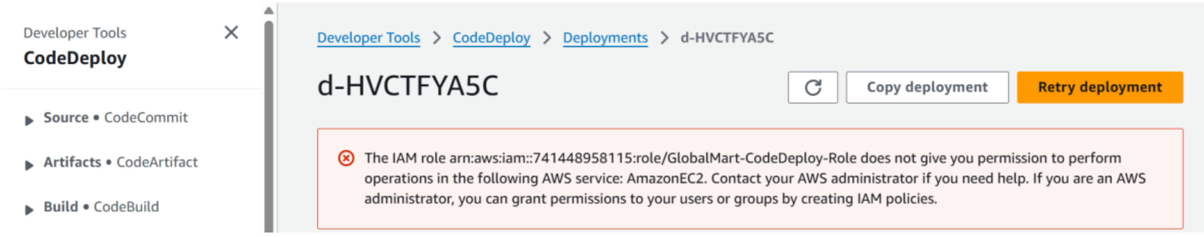
Diagnosis:

I checked CodeDeploy deployment logs:

- Went to CodeDeploy > Deployments
- Selected the failed deployment



- Reviewed error details — confirmed permission issues



I verified IAM role permissions:

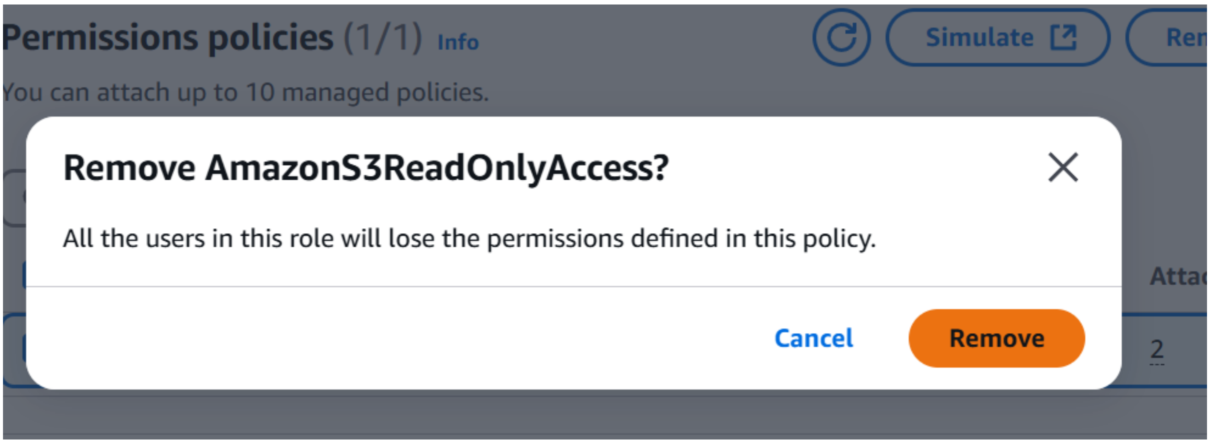
- Went to IAM > Roles > GlobalMart-CodeDeploy-Role
- Confirmed the proper policies were missing

Solution:

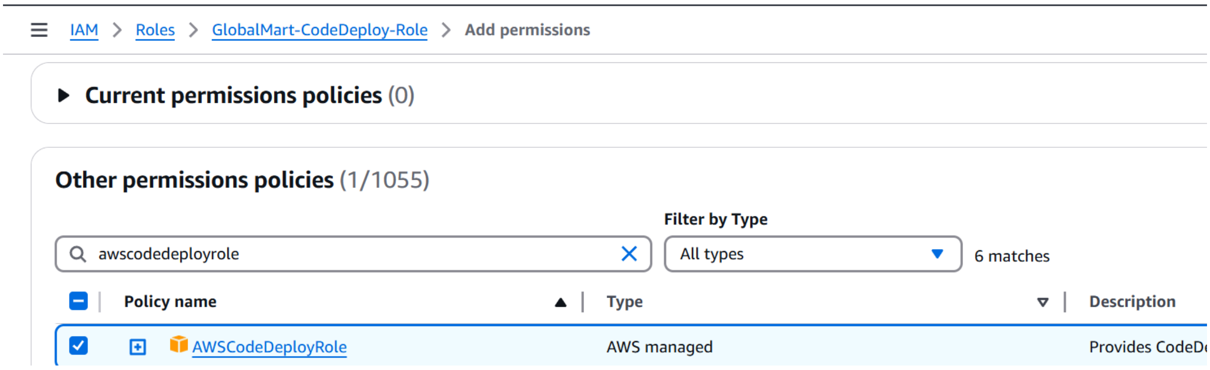
I updated the IAM role:

- Went to IAM > Roles > GlobalMart-CodeDeploy-Role

- Removed the restrictive AmazonS3ReadOnlyAccess policy

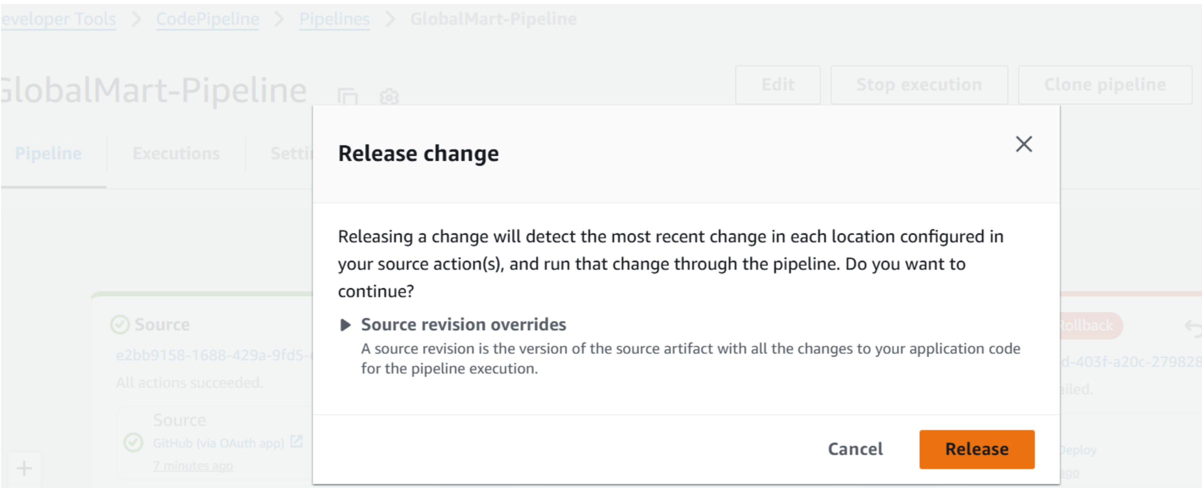


- Clicked "Add permissions" > "Attach policies"
- Searched for and attached the AWSCodeDeployRole policy

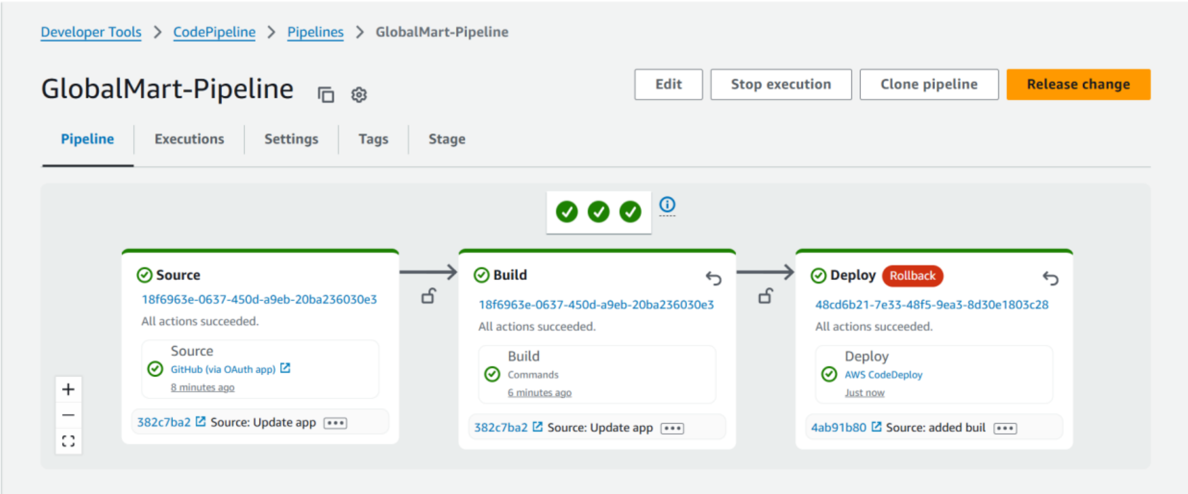


I tested the fix:

- In CodePipeline, clicked "Release change"



- Monitored the deployment progress — permission errors were resolved



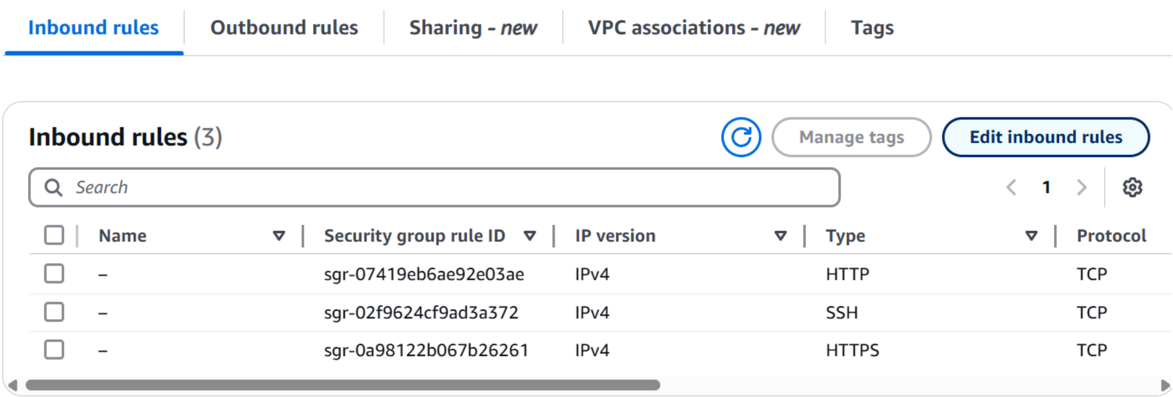
Issue 3: Security Group Configuration

Breaking the Pipeline

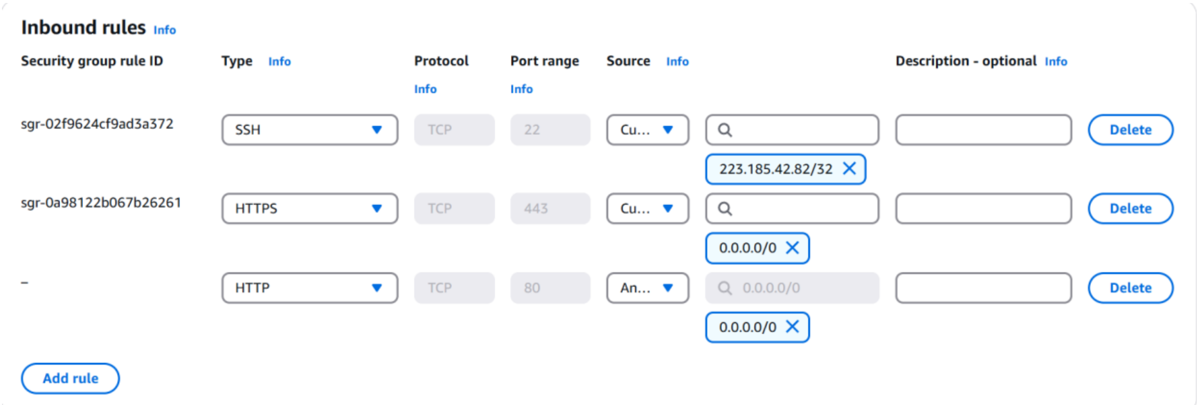
I navigated to EC2 in the AWS Console:

- Went to EC2 > Security Groups
- Found the security group attached to my EC2 instance
- Selected the security group > "Inbound rules" tab

- **Clicked "Edit inbound rules"**



- ◦ **Deleted the rule allowing HTTP (port 80) traffic**



- ◦ **Clicked "Save rules"**

I tested the security group issue:

- ◦ Tried to access the application at http://your-ec2-public-ip
- ◦ The connection timed out — confirmed the issue
- ◦ Noted that SSH access still worked, allowing remote troubleshooting

Symptoms:

- EC2 instance was running but not accessible via HTTP
- Browser connection timed out when trying to access the site

Diagnosis:

I checked EC2 instance status:

- Verified instance was running in the EC2 dashboard
- Tested SSH connection — confirmed basic connectivity

I checked security groups:

- Selected my EC2 instance > "Security" tab
- Reviewed inbound rules — confirmed HTTP rule was missing

Solution:

I updated the security group:

- Went to EC2 > Security Groups
- Selected the security group attached to my instance
- Clicked "Edit inbound rules"
- Added a new rule:
  - Type: HTTP
  - Protocol: TCP
  - Port range: 80
  - Source: 0.0.0.0/0
  - Description: "Allow HTTP traffic"
- **Clicked "Save rules"**

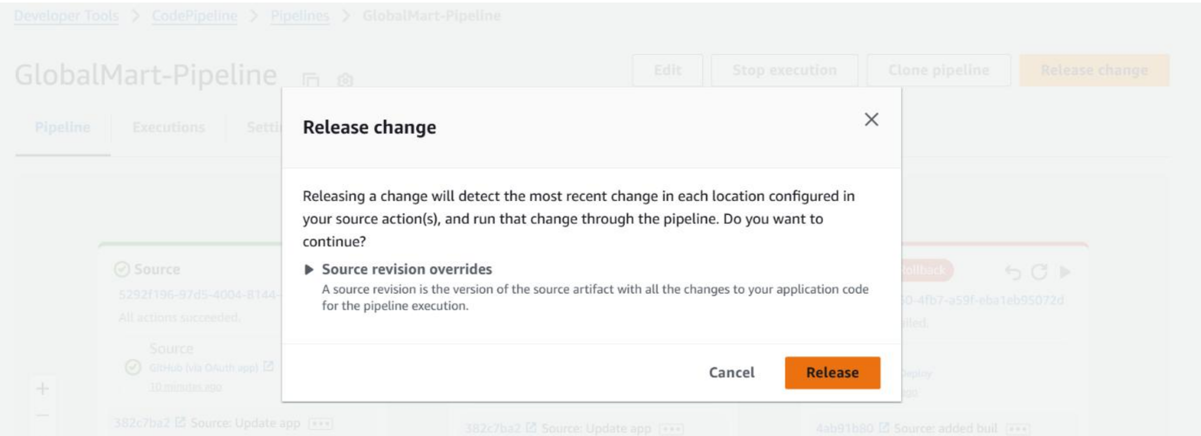
I tested the fix:

- Opened the browser and went to http://your-ec2-public-ip
- The website was now accessible

Issue 4: CodeDeploy Agent

Breaking the Pipeline

1. I connected to my EC2 instance via SSH:
    - ssh -i your-key.pem ec2-user@my-public-ip
2. I stopped the CodeDeploy agent:
    - sudo service codedeploy-agent stop
3. I tested the agent issue:
    - In CodePipeline, clicked "Release change"



    - The deployment failed and timed out
    - In CodeDeploy, confirmed the agent was not responding

Symptoms:

- Deployment failed with agent communication errors
- CodeDeploy showed timeouts when trying to reach the instance

Diagnosis:

1. I checked agent status on EC2:
    - ssh -i your-key.pem ec2-user@my-public-ip
    - sudo service codedeploy-agent status

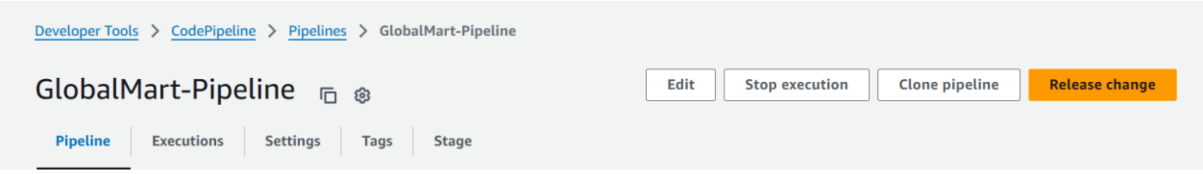
- Confirmed the agent was stopped

Solution:

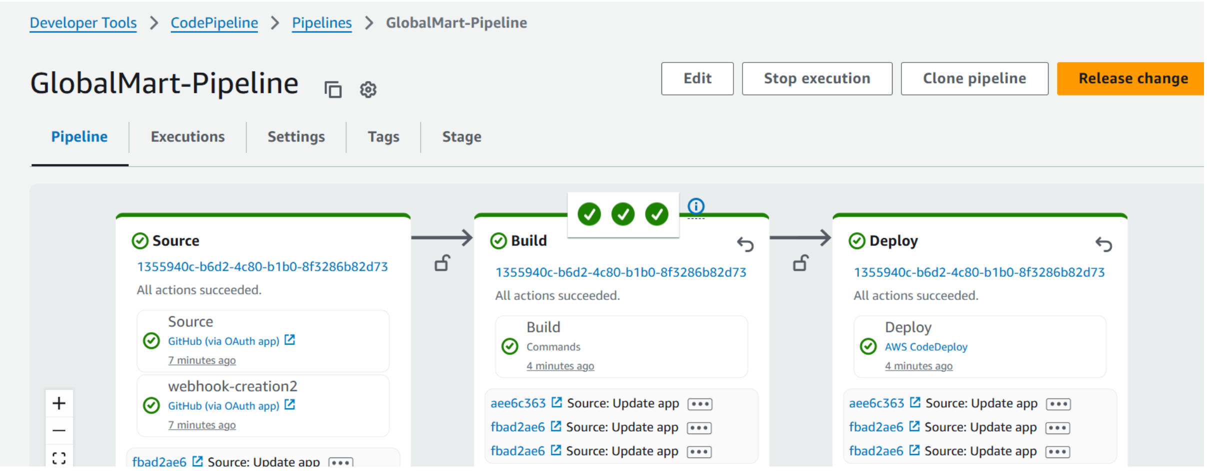
1. I started the CodeDeploy agent:
   - sudo service codedeploy-agent start
2. I verified the agent was running:
   - sudo service codedeploy-agent status
3. I configured auto-restart on reboot:
   - sudo chkconfig codedeploy-agent on

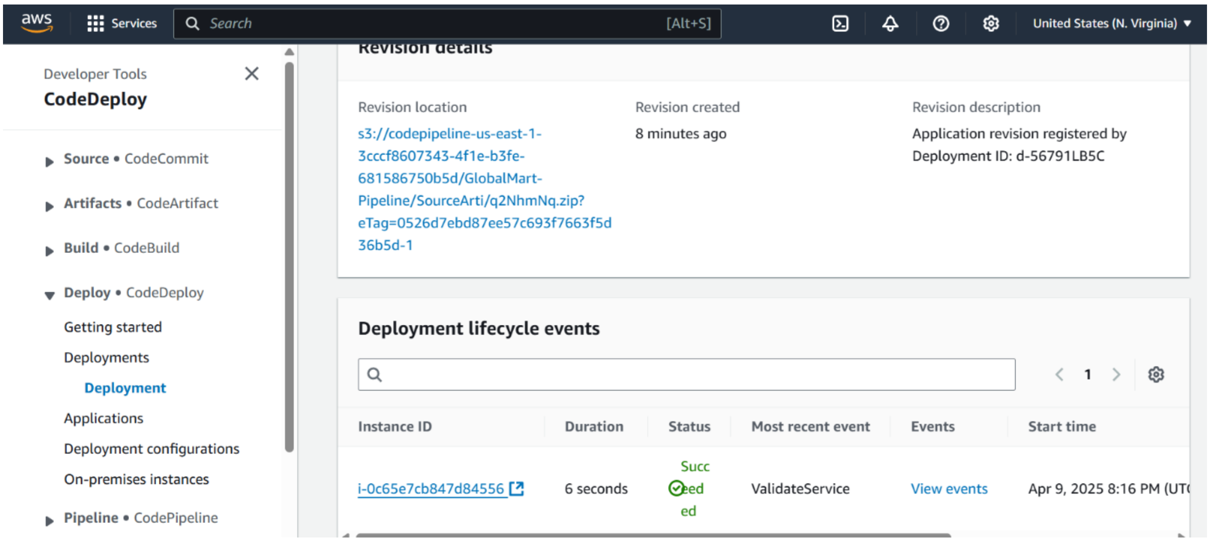
This ensured the CodeDeploy agent automatically starts whenever the instance reboots, adding resilience to the deployment pipeline.

4. I tested the fix:
   - In CodePipeline, clicked "Release change"



   - The deployment communicated properly with the agent and deployed successfully
   -




aws
Services
Search
[Alt+S]
United States (N. Virginia)
Revision details
Developer Tools
CodeDeploy
Source • CodeCommit
Artifacts • CodeArtifact
Build • CodeBuild
Deploy • CodeDeploy
Getting started
Deployments
Deployment
Applications
Deployment configurations
On-premises instances
Pipeline • CodePipeline
Revision location
s3://codepipeline-us-east-1-3cccf8607343-4f1e-b3fe-681586750b5d/GlobalMart-Pipeline/SourceArti/q2NhmNq.zip?eTag=0526d7ebd87ee57c693f7663f5d36b5d-1
Revision created
8 minutes ago
Revision description
Application revision registered by Deployment ID: d-56791LB5C
Deployment lifecycle events
1
Instance ID
Duration
Status
Most recent event
Events
Start time
i-0c65e7cb847d84556
6 seconds
Succeeded
ValidateService
View events
Apr 9, 2025 8:16 PM (UTC